SOFTWAREENTWICKLUNG / BENUTZERHANDBÜCHER

ArquitecturaRepo

Vollständiges Benutzerhandbuch

VERSION 1.4 / DEUTSCH

Acht Kapitel: Einführung, Installation, Adoption, Betrieb, Änderungen, Wiederherstellung, Abnahme und Vereinsbeispiel.

MATÍAS ACUÑA · NOVEMBER 2026

An den Quellen des ArquitecturaRepo-Starters geprüfte Anleitung.

Inhaltsverzeichnis

00 — Bevor Sie beginnen **5**
Orientierung . . . 5
1. Was ArquitecturaRepo tatsächlich liefert . . . 5
2. Warum die Arbeitsweise anders ist als ein gewöhnlicher Chat . . . 6
3. Zuständigkeiten und Entscheidungsrechte . . . 6
4. Verlässliche Informationen und dauerhafte Zustände . . . 7
5. Die drei Ebenen eines Projekts . . . 8
6. Ein sinnvoller Einstieg ohne unnötige Komplexität . . . 8
7. Die acht Kapitel dieses Handbuchs . . . 9
8. Fragen, die der Auftraggeber beantworten muss . . . 9
9. Ein zusammenhängendes Beispiel . . . 10
10. Erfolgskriterien und Grenzen . . . 10
Quellen und Gültigkeit . . . 11

01 — Installation und Einrichtung **12**
Kapitelübersicht . . . 12
1. Die notwendige Arbeitsumgebung . . . 12
2. Starter, Fork und eigentliche Anwendung . . . 13
3. Sichere Prüfung vor einer Änderung . . . 14
4. Passendes Installationsprofil wählen . . . 14
5. Die Steuerungsebene anlegen . . . 15
6. Prüfen, ohne einen Erfolg zu erfinden . . . 16
7. Die erste Bestandsaufnahme . . . 16
8. Bestehende Projekte behutsam übernehmen . . . 17
9. Zugangsdaten, Anbieter und Feedback . . . 17
10. Fehlerdiagnose und Abschlusskontrolle . . . 17
Quellen und Gültigkeit . . . 18

02 — Planner und Projektübernahme **19**
In diesem Kapitel . . . 19
1. Warum der Gesprächsverlauf nicht der Projektzustand ist . . . 19
2. Der erste lesende Rundgang . . . 20
3. Produktfragen und technische Fragen trennen . . . 20
4. Bestätigte Anforderungen erstellen . . . 21
5. Technische Grundlage und Lieferplan . . . 22
6. Adoption als überprüfbare Zustandsänderung . . . 22
7. Der Test mit einem neuen Kontext . . . 23
8. Den ersten Entwicklungsauftrag abgrenzen . . . 23
9. Ein tatsächliches Erstgespräch . . . 24
10. Häufige Fehler und Abnahme des Baselines . . . 24
Quellen und Gültigkeit . . . 25

03 — Täglicher Betrieb und kontrollierte Umsetzung **26**
Kapitelübersicht . . . 26
1. Einen Arbeitstag ohne Neustart beginnen . . . 26
2. Von einer Produktabsicht zur überprüfbaren Fähigkeit . . . 27
3. Die operative Verantwortung von I3 . . . 27
4. Externe und reservierte Freigaben . . . 28

5. Ergebnisse eines Ausführenden kritisch lesen . . . 28
6. Git, Branches und technische Kontinuität . . . 29
7. Die tägliche Fünffragenkontrolle . . . 29
8. Ein vollständiger Tagesablauf . . . 30
9. Ein klar begrenzter Arbeitsauftrag . . . 30
10. Zustände und Probleme im Controller . . . 30
11. Bericht, Prüfung und Rückroute . . . 31
12. Die Sitzung rekonstruierbar beenden . . . 31
Quellen und Gültigkeit . . . 32

04 — Produktänderungen richtig anfordern 33
Kapitelübersicht . . . 33
1. Mit dem Ergebnis beginnen . . . 33
2. Ein kompakter Änderungsauftrag . . . 34
3. Welche Entscheidungen beim CLIENT bleiben . . . 34
4. Änderungen der Meinung ohne Geschichtsverlust . . . 35
5. Qualitätswünsche ohne Technikvorgabe . . . 35
6. Prioritäten setzen und den Umfang begrenzen . . . 36
7. Wann der Planner wirklich zurückfragen muss . . . 36
8. Fünf gut formulierte Beispiele . . . 36
9. Neue Produktabsicht oder gewöhnlicher Fehler? . . . 37
10. Wenn Software funktioniert, aber nichts nützt . . . 37
11. Eine praktische Gesprächsvorlage . . . 38
Quellen und Gültigkeit . . . 38

05 — Fehler, Zwischenfälle und sichere Wiederherstellung 39
Kapitelübersicht . . . 39
1. Beweise vor Maßnahmen sichern . . . 39
2. Umgebungs- und Kontoprobleme . . . 40
3. Fehler der installierten Steuerungsebene . . . 40
4. Git ist keine allgemeine Rückgängig-Taste . . . 40
5. Wiederkehrende Fehler systemisch behandeln . . . 41
6. Sicherheit, Datenschutz und Rechte Dritter . . . 41
7. Nachweisbare Rückkehr zum Betrieb . . . 42
8. Drei vollständige Wiederherstellungsfälle . . . 42
9. Entscheidungsmatrix für sensible Operationen . . . 43
10. Was grüne Validierung nicht aussagt . . . 43
11. Feedback und Telemetrie des Starters . . . 44
Quellen und Gültigkeit . . . 44

06 — Fähigkeiten abnehmen und nachvollziehbar abschließen 45
Kapitelübersicht . . . 45
1. Eine Fähigkeit ist ein benutzbares Ergebnis . . . 45
2. Fünf Bausteine eines Abschlusses . . . 46
3. Der genaue Kandidat als Prüfgegenstand . . . 46
4. Evidenzlücken ohne künstliche neue Kampagne behandeln . . . 47
5. Annehmen, fortsetzen oder das Ziel ändern . . . 47
6. Regelmäßige Prüfung und vernünftige Stoppregel . . . 48
7. Ein überprüfbarer Abschlussdatensatz . . . 48
8. Zurückweisung und Korrektur im bestehenden Auftrag . . . 49
9. Brauchbare Software und grünes Framework . . . 49
10. Wann Arbeit wirklich beendet werden kann . . . 50
Quellen und Gültigkeit . . . 50

07 — Durchgearbeitetes Beispiel: Vereinsverwaltung 51
Der Weg durch das Beispiel 51
1. Das Produkt und seine tatsächlichen Nutzer 51
2. Fragen, die die Lösung verändern 52
3. Den Starter sicher installieren 52
4. Erste Fähigkeit: Mitglieder verwalten 53
5. Vom Planner zur ausführbaren Kampagne 54
6. Fachliche und technische Tests 54
7. Zweite Fähigkeit: Vereinsbeiträge 55
8. Das Gesamtprodukt bewerten 55
9. Erster Entwicklungszyklus mit fiktiven Daten 56
10. Einen Befund an die richtige Rolle routen 56
11. Zweiter Zyklus mit klaren Abhängigkeiten 57
12. Übung: Fortsetzung in einem neuen Kontext 57
Weiterführende Quellen 57

00 — Bevor Sie beginnen

ArquitecturaRepo · Vollständiges Benutzerhandbuch · Ausgabe v1.4 · Deutsch

> Dieses Kapitel erklärt den Zweck, die Zuständigkeiten und die Grenzen des Systems. Es ergänzt die veröffentlichte Version v1.4.0. Die Beispiele beschreiben Arbeitsweisen, nicht eine bereits abgenommene Anwendung. Maßgeblich für die Installation sind der aktuelle öffentliche Einstieg und die Versionshinweise.

Orientierung

1. Was ArquitecturaRepo tatsächlich liefert
2. Warum die Arbeitsweise anders ist als ein gewöhnlicher Chat
3. Zuständigkeiten und Entscheidungsrechte
4. Verlässliche Informationen und dauerhafte Zustände
5. Die drei Ebenen eines Projekts
6. Ein sinnvoller Einstieg ohne unnötige Komplexität
7. Die acht Kapitel dieses Handbuchs
8. Fragen, die der Auftraggeber beantworten muss
9. Ein zusammenhängendes Beispiel
10. Erfolgskriterien und Grenzen

1. Was ArquitecturaRepo tatsächlich liefert

ArquitecturaRepo ist eine portable Ausgangsstruktur für die kontrollierte Entwicklung von Software. Sie enthält dokumentierte Rollen, Entscheidungsregeln, wiederverwendbare Vorlagen, Installationsprofile, prüfbare Datenverträge sowie Python-Werkzeuge für Validierung und Wiederaufnahme. Ein optionaler lokaler Kampagnencontroller kann bestimmte Arbeitszustände verwalten. Diese Teile ergeben gemeinsam einen Entwicklungsrahmen, aber keine fertige Anwendung und keine Garantie für eine autonome Softwarefabrik.

Nach der Installation steht eine organisatorische und technische Steuerungsebene bereit. Sie legt fest, welche Informationen für ein Projekt dauerhaft festgehalten werden müssen, wie sich eine neue Sitzung orientiert und wann ein Ergebnis als nachgewiesen gelten darf. Das konkrete Produkt muss weiterhin verstanden, entworfen, implementiert, getestet und von den zuständigen Personen akzeptiert werden. Die Installation eines Frameworks ersetzt keine dieser Arbeiten.

Die Unterscheidung ist wichtig: Ein erfolgreicher Test des Starters beweist, dass ein Mechanismus des Starters unter den getesteten Bedingungen funktioniert. Er beweist nicht, dass eine damit entwickelte Kundenanwendung korrekt, sicher oder angenehm zu bedienen ist. Ebenso ist ein synthetisches Referenzprojekt ein technischer Prüfstein, kein realer Nutzerbeleg. Wenn ein tatsächlicher Nachweis fehlt, muss der Zustand als unbekannt oder blockiert erscheinen.

Die öffentliche Version v1.4.0 wurde als GitHub-Release mit einem portablen ZIP, Prüfsumme und Manifest bereitgestellt. Die veränderliche Hauptentwicklungszweig kann spätere Korrekturen und diese ausführlichen Handbücher enthalten. Eine Datei

auf main ist deshalb nicht automatisch Teil des unveränderlich benannten Release-Pakets. Vergleichen Sie beim Installieren die genaue Version, die erwartete SHA-256-Prüfsumme und die tatsächlich verwendeten Dateien.

Das System kann ohne bezahlte KI-Dienste eingeführt werden. Es verlangt weder einen bestimmten Modellanbieter noch einen vorgeschriebenen Programmieragenten. Die Namen CLIENT, Planner, Controller und Spezialist bezeichnen Entscheidungsbefugnisse und Verantwortlichkeiten. Ob diese Funktionen durch Menschen, Werkzeuge oder ausdrücklich genehmigte Agenten wahrgenommen werden, ist eine eigene Entscheidung des jeweiligen Projekts.

2. Warum die Arbeitsweise anders ist als ein gewöhnlicher Chat

Ein Chat kann schnelle Ergebnisse liefern. Man beschreibt eine Funktion, erhält Code, testet ihn und bittet um Korrekturen. Für ein kleines Experiment ist das oft völlig ausreichend. Schwierigkeiten entstehen, wenn Anforderungen im Verlauf vieler Nachrichten geändert werden, mehrere Bearbeiter denselben Stand unterschiedlich verstehen oder eine neue Sitzung den früheren Kontext nicht mehr kennt. Dann muss der Auftraggeber wiederholt erklären, was bereits beschlossen wurde und welcher Code tatsächlich maßgeblich ist.

ArquitecturaRepo versucht dieses Problem durch rekonstruierbaren Zustand zu reduzieren. Die aktuelle Anforderung, die bestätigte technische Grundlage, die gültige Arbeitsfreigabe, die Identität des Kandidaten und seine Nachweise liegen in nachvollziehbaren Repository-Oberflächen. Ein neuer Bearbeiter soll sich daran orientieren können, ohne ein langes Gespräch auswendig zu kennen. Der Nutzen entsteht nicht aus mehr Dokumentation an sich, sondern daraus, dass Entscheidungen und Beweise auffindbar bleiben.

Das Framework macht gewöhnliche Programmierarbeit nicht automatisch schneller. Für eine winzige Einmalaufgabe kann die Einrichtung mehr Aufwand verursachen als die Aufgabe selbst. Für mehrwöchige Projekte, wiederkehrende Übergaben, mehrere technische Zuständigkeiten oder risikoreiche Änderungen kann eine klare Arbeitsführung dagegen Fehlannahmen, doppelte Arbeit und teure Wiederherstellung vermeiden. Ob das gelingt, muss am Ergebnis gemessen werden.

Ein gutes Vergleichsexperiment verwendet dieselbe kleine Produktaufgabe zweimal: einmal mit einem ad-hoc Chat, einmal mit klarer Ausgangslage, Zuständigkeiten und Übergabe. Vergleichen Sie nicht nur die Geschwindigkeit des ersten Codes, sondern auch die Korrektheit nach einer Anforderungsänderung, die Wiederaufnahme durch eine neue Sitzung und den Aufwand für eine begründete Freigabe. Beide Ansätze können sinnvoll sein; der Kontext entscheidet.

3. Zuständigkeiten und Entscheidungsrechte

Der CLIENT ist der Auftraggeber und Eigentümer der Produktabsicht. Diese Person entscheidet, welches Problem gelöst werden soll, welches Verhalten gewünscht ist, welche Nutzer betroffen sind und welche Qualität sich richtig anfühlt. Ausgaben, externe Veröffentlichung, Nutzung personenbezogener Daten und andere vorbehaltene Entscheidungen bleiben ebenfalls bei der zuständigen Person. Sie soll nicht nebenbei Merge-Konflikte beheben oder unklare Testprotokolle entziffern müssen.

Die interne Struktur kennt vier Rollenklassen, ohne vier unabhängige Programme zu verlangen:

Rolle	Kernverantwortung	Typische Grenze
I1 Master Owner, bei Bedarf	Gemeinsame Steuerung mehrerer Projekte, Kapazität und systemische Konsistenz	Nicht jede kleine technische Aufgabe selbst planen
I2 Planner	Anforderungen interpretieren, Architektur und Lieferumfang bestimmen, strategische Annahme prüfen	Nicht jeden Programmierfehler als neue Planungsschleife behandeln
I3 Execution Controller	Umsetzung, Debugging, Wiederholungen, Tests, Integration, Wiederherstellung und prüfbare Kandidaten	Keine neue Produktabsicht erfinden oder eigene Arbeit endgültig genehmigen
I4 Fachkraft oder Reviewer	Abgegrenzte Implementierung, Recherche, Tests oder unabhängige Prüfung	Keine übergeordneten Freigaben ersetzen

In einem kleinen Projekt können mehrere Funktionen von derselben technischen Person übernommen werden. Entscheidend ist, dass die Befugnisse in jeder Situation klar bleiben. Eine unabhängige Prüfung erfordert jedoch tatsächlich einen Blick außerhalb der eigenen Implementierungsperspektive. Ein zusätzlicher Chat mit neuem Namen ist nicht automatisch eine unabhängige Prüfung.

Der Planner soll die ganze gewünschte Fähigkeit verstehen und einen sinnvollen Auftrag formulieren. Ist der technische Auftrag freigegeben, führt I3 die alltägliche Arbeit fort: Fehler analysieren, innerhalb des Auftrags korrigieren, Regressionen testen, Zwischenstände konsolidieren und eine überprüfbare Endfassung vorlegen. Ein fehlgeschlagener Unit-Test löst normalerweise keine neue Kampagne und keine neue strategische Grundsatzdiskussion aus.

Bei einer echten Produktänderung kehrt die Entscheidung dagegen zum CLIENT und, soweit strategisch nötig, zum Planner zurück. Ein Beispiel: Die Forderung, eine lokale Mitgliederverwaltung künftig als öffentlich erreichbaren Internetdienst zu betreiben, ändert Zugriffskontrolle, Betrieb, Datenschutz und wahrscheinlich die technische Architektur. Das ist keine gewöhnliche Fehlerkorrektur.

4. Verlässliche Informationen und dauerhafte Zustände

Gespräche sind nützlich für Erklärungen und Entscheidungen, aber ein Chatprotokoll ist kein zuverlässiges Inventar des aktuellen Codes. Entscheidend sind die tatsächlich vorhandenen Dateien, der konkrete Git-Stand, die relevante Datenbank, vorhandene Artefakte und gültige Zustandsnachweise. Danach kommen die ausdrücklich bestätigten Anforderungen und die aktuelle Arbeitsfreigabe. Ältere Zusammenfassungen liefern Kontext, ersetzen jedoch keine Überprüfung.

Dabei gilt eine wichtige Ausnahme: Die neueste ausdrückliche Produktentscheidung des CLIENT kann ältere Anforderungen aufheben. Diese Entscheidung muss dann

sauber in die aktuelle Produktbeschreibung übernommen werden. Ein technisches System darf nicht stillschweigend an einer früheren Präferenz festhalten, nur weil sie bereits in einem Dokument steht. Umgekehrt darf eine KI nicht aus einem späteren eigenen Bericht eine neue Freigabe konstruieren.

Eine neue Arbeitssitzung beginnt mit einem Misstrauensvorbehalt gegenüber ihrem eigenen Kontext. Der Bearbeiter ermittelt, welche Repository-Version gilt, welches Produkt gemeint ist, welche Anforderungen bestätigt wurden und welche Kampagne ausgeführt werden darf. Kann ein für die nächste Änderung wesentliches Detail nicht nachgewiesen werden, wird die abhängige Änderung angehalten. Unabhängige, lesende Untersuchungen dürfen fortgesetzt werden.

Der erzeugte Steuerungsordner heißt normalerweise .agentic-sdlc/. Ein Status wie HOLD oder UNTRUSTED_CONTEXT direkt nach der Installation bedeutet nicht, dass etwas kaputt ist. Er verhindert lediglich, dass nicht überprüfte Beispielwerte automatisch zu gültigen Projektentscheidungen werden. Erst eine saubere Bestandsaufnahme und Adoption machen daraus ein projektspezifisches System.

5. Die drei Ebenen eines Projekts

Für den Einstieg helfen drei Ebenen. L0 enthält belegbare Aussagen und Entscheidungen des Auftraggebers einschließlich späterer Klarstellungen. L1 hält die bestätigte Interpretation als aktuelle Anforderungen, Produktmodell und Product State fest. L2 enthält technische Umsetzung, Quellcode, Datenbanken, Kampagnen, Testergebnisse und Laufzeitnachweise.

Die Ebenen erklären, weshalb ein schönes Konzept noch keine fertige Funktion ist. Eine L1-Anforderung kann gut beschrieben und freigegeben sein, obwohl die entsprechende L2-Implementierung fehlt. Ebenso kann ein Entwickler eine technische Funktion fertigstellen, die nicht zur bestätigten L0-Produktabsicht passt. Ein grüner Test auf L2 repariert diesen Bedeutungsfehler nicht.

Verwechseln Sie auch die Worte Zustand und Wahrheit nicht. Eine Datei namens CURRENT kann veraltet sein, wenn sie auf den falschen Git-Stand oder eine frühere Datenbank verweist. „Aktuell“ ist eine überprüfbare Beziehung zwischen Referenzen und realen Artefakten. Wenn diese Beziehung gebrochen ist, muss sie rekonstruiert werden, bevor eine risikoabhängige Freigabe erfolgt.

6. Ein sinnvoller Einstieg ohne unnötige Komplexität

Wählen Sie das kleinste geeignete Profil. Für ein einfaches Lernprojekt genügt häufig SMALL. Beginnen Sie mit einer lesenden Bestandsaufnahme und installieren Sie die Steuerungsebene zunächst nur in einem dafür geeigneten Zielprojekt. Ein bereits streng geregeltes Projekt braucht vor jeder Änderung eine eigene Attachment-Prüfung, weil vorhandene Verantwortlichkeiten nicht automatisch überschrieben werden dürfen.

Vor dem ersten größeren Auftrag sammeln Sie einen kurzen Produktsteckbrief: Zielgruppe, Hauptproblem, wichtigste Nutzerreise, benötigte Daten, Betriebsumgebung, Grenzen, geplanten Termin und offene Fragen. Anschließend rekonstruiert die technische Rolle den bestehenden Quellstand, legt eine nachvollziehbare technische Grundlage fest und plant einen kleinen, in sich prüfbaren Lieferabschnitt.

Für diese Einführung müssen Sie keine acht Kapitel auswendig lernen. Das erste echte

Erfolgskriterium lautet: Eine neue Sitzung kann die Ausgangslage und die nächste zulässige Handlung aus dem Repository nachvollziehen. Wenn dies nicht möglich ist, sollte nicht einfach mehr Code produziert werden.

7. Die acht Kapitel dieses Handbuchs

Kapitel	Gegenstand	Praktisches Ergebnis
00	Einführung und Entscheidungsrechte	Sie kennen Zweck und Grenzen.
01	Installation	Das richtige Paket ist überprüft und sicher eingerichtet.
02	Planner und Adoption	Anforderungen und Ausgangszustand sind nachvollziehbar.
03	Täglicher Betrieb	Die Umsetzung bleibt innerhalb einer gültigen Arbeitsfreigabe.
04	Änderungswünsche	Neue Produktabsichten werden präzise und ohne technische Mikromanagementpflicht formuliert.
05	Fehler und Wiederherstellung	Belege bleiben erhalten, Risiken werden eingegrenzt und die Fortsetzung ist sicher.
06	Abschluss von Fähigkeiten	Tests, Integrationsnachweise und menschliche Annahme werden getrennt geprüft.
07	Durchgearbeitetes Vereinsbeispiel	Der ganze Weg ist an einer konkreten Funktion sichtbar.

Die acht kurzen mehrsprachigen Anleitungen und die zugehörigen FAQs erleichtern den Einstieg. Sie sind keine Ersatzfassung dieses ausführlichen Buches. Das historische spanische Handbuch v1.3 wird aus Gründen der Nachvollziehbarkeit separat aufbewahrt. Bei widersprüchlichen Versionsangaben gilt der tatsächlich installierte Stand samt zugehöriger Dokumentation.

8. Fragen, die der Auftraggeber beantworten muss

Beschreiben Sie zunächst, was sich aus Sicht eines Nutzers ändern soll. „Mitglieder verwalten“ ist ein Ausgangspunkt, aber noch keine ausreichende Entscheidung. Können Mitglieder deaktiviert werden? Wird ihre Zahlungshistorie weiter benötigt? Wer darf sensible Informationen sehen? Soll das System nur auf einem Rechner laufen oder später von mehreren Orten erreichbar sein? Die Antworten sind Produktentscheidungen.

Die technischen Mittel müssen Sie meist nicht vorgeben. Ob für eine kleine Offline-Anwendung SQLite oder für einen Mehrbenutzerdienst PostgreSQL sinnvoller ist, soll die technische Planung anhand echter Anforderungen begründen. Sie können eine Technologie als ausdrückliche Randbedingung vorgeben, müssen aber nicht die Entwurfsarbeit der Entwickler übernehmen.

Wenn Sie eine Antwort noch nicht kennen, schreiben Sie „offen“. Der Planner ermittelt die Folgen und fragt nur nach Entscheidungen, die das Ergebnis tatsächlich verändern. Aus einer ungeklärten Frage darf keine scheinbar bestätigte Regel werden. Das schützt Sie zugleich vor endlosen Rückfragen über technische Einzelheiten.

9. Ein zusammenhängendes Beispiel

Ein lokaler Sportverein möchte seine Mitglieder und Beiträge verwalten. Die Verwaltungsmitarbeiterin braucht eine übersichtliche Suchfunktion, die Anmeldung neuer Mitglieder und den Verlauf bezahlter Beiträge. Der CLIENT bestätigt, dass die erste Version keine Online-Zahlungen anbietet und dass die Daten auf einem lokalen Rechner verbleiben. Damit sind bereits mehrere wichtige Grenzen geklärt.

Der Planner prüft zunächst, ob eine Anwendung existiert und welche Daten bereits gespeichert werden. Anschließend beschreibt er eine erste Fähigkeit, beispielsweise die sichere Aufnahme eines Mitglieds mit Pflichtfeldprüfung und eindeutiger Identität. Die Anforderungen werden vom CLIENT bestätigt, die technische Grundlage und die Prüfbedingungen festgehalten und der Entwicklungsauftrag freigegeben.

I3 organisiert danach Implementierung, Tests und gegebenenfalls eine unabhängige Prüfung. Ein Test findet einen Fehler bei doppelt angelegten Mitgliedern. I3 behebt ihn innerhalb desselben Auftrags und weist die Korrektur erneut nach. Erst ein konkreter, überprüfter Kandidat wird zur strategischen und menschlichen Annahme vorgelegt. Der Auftraggeber prüft, ob die Oberfläche verständlich ist und der gewünschte Arbeitsablauf tatsächlich funktioniert.

Angenommen, der Verein verlangt später Zugriff über das Internet. Die frühere Entscheidung „nur lokal“ wird nicht heimlich ersetzt. Die Änderung benötigt neue Produktfreigabe und eine Bewertung der Auswirkungen auf Authentifizierung, Hosting, Backups, Datenschutz und Betrieb. Die bereits bewährte lokale Funktion kann erhalten bleiben, aber die Freigabe ihrer Internetnutzung folgt anderen Gates.

10. Erfolgskriterien und Grenzen

Nach zwei oder drei Entwicklungsabschnitten sollten Sie einen neuen Bearbeiter bitten können, ohne alte Chats zu lesen folgende Fragen zu beantworten: Welches Produkt wird gebaut? Welche Anforderungen gelten? Welche Fähigkeit funktioniert bereits? Wo liegt ihr überprüfbarer Kandidat? Welche Risiken sind noch offen? Welche nächste Handlung ist tatsächlich erlaubt?

Ein gutes Ergebnis ist eine nutzbare Fähigkeit mit glaubwürdigen Nachweisen und klarer Fortsetzung. Viele Commits, hübsche Berichte, lange Laufzeiten und zusätzliche Agenten sind höchstens Mittel zum Zweck. Wenn sich damit weder nutzbare Software noch nachweisbare Risikoreduktion verbinden lassen, muss die Arbeitsweise überprüft werden.

Das Framework kann Fehler sichtbar und Übergaben zuverlässiger machen. Es kann nicht garantieren, dass eine Software ohne fachliche Entscheidungen, Datenschutzprüfung, echte Integration oder menschliche Qualitätseinschätzung

fertig wird. Diese Grenzen gehören zur ehrlichen Produktbeschreibung und müssen auch in einer überzeugend gestalteten Dokumentation deutlich bleiben.

Weiter: Kapitel 01, Installation.

Quellen und Gültigkeit

- Veröffentlichte Version v1.4.0
- Öffentlicher Einstieg und Support
- Versionshinweise und Upgrade-Grenzen
- Architekturübersicht
- Verantwortungsmodell
- MIT-Lizenz

Diese Hinweise beschreiben den Starter. Eine reale Anwendung erfordert eigene Nachweise und Freigaben.

01 — Installation und Einrichtung

> Dieses Kapitel führt durch die Installation des öffentlichen Starters und erklärt die Grenze zur echten Übernahme in ein Projekt. Aktuelle Befehle und Unterstützungsgrenzen stehen in der v1.4-Anleitung. Eine erfolgreiche Installation bedeutet ausdrücklich nicht, dass die Anwendung verstanden, entwickelt oder abgenommen wurde.

Kapitelübersicht

1. Die notwendige Arbeitsumgebung
2. Starter, Fork und eigentliche Anwendung
3. Sichere Prüfung vor einer Änderung
4. Passendes Installationsprofil wählen
5. Die Steuerungsebene anlegen
6. Prüfen, ohne einen Erfolg zu erfinden
7. Die erste Bestandsaufnahme
8. Bestehende Projekte behutsam übernehmen
9. Zugangsdaten, Anbieter und Feedback
10. Fehlerdiagnose und Abschlusskontrolle

1. Die notwendige Arbeitsumgebung

Die folgenden Beispiele benutzen Windows, PowerShell, Git und Python. Die Architektur ist nicht auf diese Kombination beschränkt. Sie können ein anderes Betriebssystem, einen anderen Git-Dienst oder eine eigene Entwicklungsumgebung verwenden. Wesentlich sind eine erreichbare Quellbasis, nachvollziehbare Versionen, geeignete Dateirechte und die Möglichkeit, die mitgelieferten Prüfwerkzeuge auszuführen.

Für den portablen v1.4-Starter wurde Python 3.12 in der öffentlichen Continuous Integration getestet. Kontrollieren Sie zunächst die tatsächlich aktive Version. In PowerShell können Sie beispielsweise ausführen:

```
& "python" --version
& "py" --version
```

Falls beide Befehle scheitern, prüfen Sie, ob Python installiert und über den aktuellen Terminalpfad erreichbar ist. Installieren Sie nur aus einer vertrauenswürdigen Quelle. Ein fehlgeschlagener Befehl ist kein Grund, Dateien im Anwendungsprojekt vorschnell zu löschen. Halten Sie die Fehlermeldung und den Exitcode fest und untersuchen Sie die Ursache.

Sie benötigen keine eigene leistungsfähige Grafikkarte, wenn ein ausdrücklich freigegebener externer Programmierdienst die Rechenarbeit übernimmt. Umgekehrt zwingt Sie das Framework zu keinem solchen Dienst. Der lokale Starter selbst setzt keine automatische Modellanmeldung voraus. Lassen Sie die Frage, ob ein KI-Werkzeug später eingesetzt werden soll, zunächst getrennt von der Installation.

Kontrollieren Sie außerdem Speicherplatz und den geplanten Zielort. Quellcode, abgeleitete Artefakte, Cache-Verzeichnisse und zusätzliche Arbeitsbäume können sich unbemerkt vervielfachen. Verwenden Sie nach Möglichkeit einen dauerhaften Checkout pro Repository und räumen Sie temporäre Ausgaben auf, wenn sie nicht mehr als gültiger Nachweis gebraucht werden. Eine Kopie „für alle Fälle“ ist kein belastbarer Grund für zusätzlichen Speicherverbrauch.

2. Starter, Fork und eigentliche Anwendung

Das häufigste Missverständnis besteht darin, dass jemand ArquitecturaRepo öffnet und anschließend einen Agenten bittet, dort seine Kundenanwendung zu programmieren. Das Ausgangsrepository enthält aber die allgemeinen Protokolle, Vorlagen und Werkzeuge. Die Anwendung, die gebaut werden soll, hat normalerweise ein eigenes Repository. Die Steuerungsebene wird in diese Anwendung installiert.

Ort	Zweck	Wichtige Abgrenzung
ArquitecturaRepo	Öffentlicher Quellstand des Starters mit Protokollen und Tests	Enthält nicht automatisch Ihre Produktanforderungen
Eigener Fork, falls benötigt	Angepasste Variante des Starters	Ist noch keine Kundenanwendung
Eigenes Anwendungsrepository	Tatsächlicher Code, Daten, Konfiguration und übernommene Steuerungsebene	Darf keine erfundenen Beispielzustände übernehmen

Eine plausible lokale Ordnung ist etwa:

```
C:\Projekte\
    ArquitecturaRepo\
    verein-app\
```

Die Namen sind Beispiele. Entscheidend ist, dass das Installationswerkzeug im Starter aufgerufen wird und sein Zielparameter eindeutig auf die vorgesehene Anwendung zeigt. Die anschließend erzeugte Ordnerstruktur trägt den Namen .agentic-sdlc/. Sie ist ein organisatorischer Bestandteil der Anwendung, keine zweite Anwendung.

Wenn Sie keine dauerhaften Anpassungen am Starter vornehmen, genügt ein normaler Checkout oder ein geprüftes Release-Archiv. Für eigene Erweiterungen kann ein Fork sinnvoll sein. Kopieren Sie aber niemals den beispielhaften Product State eines synthetischen Projekts als echte Wahrheit in Ihre Anwendung. Solche Dateien enthalten absichtlich erfundene IDs, Anforderungen und Zustände, damit Tests reproduzierbar bleiben.

Beziehen Sie die veröffentlichte Ausgabe vorzugsweise aus dem v1.4.0-Release. Vergleichen Sie ZIP und veröffentlichte SHA-256-Prüfsumme und bewahren Sie die Versionsidentität nachvollziehbar auf. Der Branch main kann nach dem Release weiterentwickelte Dokumentation enthalten. Auch ein vollständig lesbares Handbuch auf main ist nicht rückwirkend Teil des früher gepackten ZIP.

3. Sichere Prüfung vor einer Änderung

Öffnen Sie das Terminal zuerst im Starter und vergewissern Sie sich, dass Sie nicht versehentlich in einem anderen Repository stehen. Drei lesende Kontrollen reichen häufig für den Anfang:

```
Get-Location
& "python" --version
& "git" status --short
```

Der erste Befehl zeigt Ihren aktuellen Ort. Der zweite prüft die Interpreterverfügbarkeit. Der dritte meldet vorhandene Git-Änderungen. Zeigt Git eine veränderte oder unversionierte Datei an, sollten Sie sie nicht einfach verwerfen, um einen vermeintlich sauberen Zustand herzustellen. Finden Sie zunächst heraus, wer sie erzeugt hat und ob sie zum aktuellen Projekt gehört.

Prüfen Sie anschließend den Zielpfad separat. Ein Agent kann eine andere Arbeitsmappe geöffnet haben als das Terminal. Ein GitHub-Desktopfenster kann den Starter anzeigen, obwohl Sie Ihre Anwendung ändern wollten. Die bloße Anmeldung an einen Dienst sagt nichts darüber aus, ob das richtige Repository, der richtige Branch und die notwendigen Rechte ausgewählt sind.

Wenn die Zielanwendung bereits über Daten, Migrationen, aktive Kampagnen, eine Steuerungsebene oder vorhandene Schutzregeln verfügt, überspringen Sie den normalen Installationsschritt. Verwenden Sie stattdessen die lesende Prüfung für bereits geregelte Projekte. Eine neue Installation darf vorhandene gültige Informationen nicht überschreiben, auch wenn der Zielordner älter aussieht als die aktuelle Version des Starters.

4. Passendes Installationsprofil wählen

Die Profile beschreiben Anforderungen des Projekts, keine Rangliste der verwendeten KI-Modelle. Die verbindliche Definition steht in profiles.json. Wählen Sie das einfachste Profil, das die nachgewiesenen Risiken sinnvoll abbildet.

Profil	Typischer Anlass
SMALL	Einfaches Projekt mit begrenzter Zustands- und Betriebsgefahr
STATEFUL	Dauerhafte Benutzerdaten, Datenbankmigrationen, Backups oder Wiederherstellung
ARTIFACT_HEAVY	Viele oder große erzeugte Dateien mit Speicher-, Herkunfts- und Lebenszyklusfragen
MULTI_WORKSTREAM	Mehrere getrennt geplante Arbeitsbereiche für ein gemeinsames Produkt
MULTI_PRODUCT	Mehrere Produkte mit gemeinsam genutzter Infrastruktur oder Regeln

Profil	Typischer Anlass
HIGH_CONSEQUENCE	Besonders wichtige Sicherheits-, Datenschutz-, Finanz- oder andere schwer rückgängig zu machende Auswirkungen

Ein kleines Lernprojekt beginnt oft sinnvoll mit SMALL. Für eine Mitgliederverwaltung, die langfristige Zahlungsdaten führt, ist STATEFUL zu prüfen. Die Anwesenheit einer Datenbank macht einen Master Owner jedoch nicht automatisch erforderlich. Eine besonders anspruchsvolle Bezeichnung erhöht weder die Sicherheit noch die Produktqualität, wenn die dazugehörigen realen Nachweise fehlen.

Notieren Sie die Gründe für das Profil kurz und überprüfbar. Wenn sich der Umfang später verändert, wird die Auswahl anhand der neuen Fakten neu bewertet. Ein Profil berechtigt nicht automatisch zu Produktionsänderungen, Datenverarbeitung oder kostenpflichtigen Diensten.

5. Die Steuerungsebene anlegen

Im geprüften v1.4-Starter können Sie für ein neues, noch nicht geregeltes Projekt beispielsweise folgenden PowerShell-Aufruf verwenden:

```
& "python" "tools\scaffold_project.py" --target "..\mein-projekt" --
profile "SMALL" --product-id "MEIN_PROJEKT" --objective "Anfangsziel; Bestätigung ausste
-language "de" --no-feedback-workflow
```

Die Parameter sind bewusst sichtbar. target bezeichnet das genaue Zielverzeichnis. profile wählt die Kontrollfamilie. product-id ist eine projektspezifische Kennung. objective ist lediglich eine vorläufige Zielbeschreibung, solange die Anforderungen noch nicht bestätigt sind. language de wählt die deutsche installierte Dokumentation und Rückmeldungen.

Für das spätere Beispiel einer Vereinsverwaltung mit langfristigen Mitglieder- und Beitragsdaten könnte die Auswahl anders aussehen:

```
& "python" "tools\scaffold_project.py" --target "..\verein-app" --
profile "STATEFUL" --product-id "VEREIN_APP" --objective "Mitglieder und Beiträge verwal
-language "de" --no-feedback-workflow
```

Die beiden Beispiele sollen nicht nacheinander auf dasselbe Verzeichnis angewendet werden. Entscheiden Sie sich für das tatsächlich passende Profil und prüfen Sie den Pfad vor dem ersten Aufruf. Der Starter soll bestehende Anwendungscodes und Arbeitsabläufe erhalten. Trifft er auf eine vorhandene kanonische Steuerungsebene, muss eine gewöhnliche Neuinstallation verweigert werden.

Ein erster Zustand HOLD / UNTRUSTED_CONTEXT ist beabsichtigt. Er besagt: Die Steuerungsebene wurde erzeugt, kennt aber die aktuellen Anforderungen und technischen Fakten Ihrer Anwendung noch nicht. Die sichere Einrichtung darf keinen erfolgreichen Product State, keine bestätigte Datenbankversion und keine gültige Entwicklungskampagne erfinden.

Die optionale Einstellung –with-runtime ergänzt Referenzmechanik für Kampagnen. Sie startet damit weder einen externen Programmieragenten noch kauft sie API-Leistungen oder bestätigt die Ausführungsfähigkeit einer Desktopanwendung. Solche

Behauptungen benötigen eigenständige Berechtigungen und Tests im tatsächlich vorgesehenen Umfeld.

6. Prüfen, ohne einen Erfolg zu erfinden

Nach erfolgreicher Einrichtung wechseln Sie in das Zielprojekt. Mit der dort installierten Steuerungsebene führen Sie die im öffentlichen Einstieg beschriebenen Prüfungen aus:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

Die erste Zeile installiert die deklarierte lokale Validierungsabhängigkeit. Die folgenden Befehle berichten erkannten Zustand, untersuchen das Projekt zunächst lesend und prüfen die Wiederherstellbarkeit des Kontexts. Ersetzen Sie diese Befehle nicht durch willkürliche Statusänderungen, wenn eine Prüfung noch HOLD meldet.

Ein erfolgreiches Scaffold hat drei einfache Belege: Der Ordner .agentic-sdlc/ liegt an der beabsichtigten Stelle, die bereits vorhandenen Anwendungsdateien sind erhalten und die Kontrollwerkzeuge lassen sich mit aussagekräftigen Ergebnissen aufrufen. Ein fehlendes Python-Paket wie jsonschema ist zuerst ein lokales Abhängigkeitsproblem. Es ist kein Anlass, eine aktuelle Anforderung oder eine Freigabe zu fälschen.

Der Aufruf discover_project ermittelt eine lesende Ausgangsbeschreibung. Er ist keine vollständige Sicherheitsprüfung Ihrer Datenbank. reconstruct_context soll sichtbar machen, welche Informationen für eine neue Sitzung noch fehlen. Dass diese Prüfung vor der Adoption keine arbeitsfähige Fortsetzung bescheinigt, ist ein korrektes Ergebnis.

Dokumentieren Sie den verwendeten Startersnapshot, die Zielumgebung und wichtige Ausgaben. Fragen Sie nicht nur, ob ein Prozess mit Exitcode null beendet wurde, sondern ob seine Aussage zum erwarteten Gate passt. Strukturvalidierung, fachliche Erfüllung und menschliche Benutzbarkeit sind verschiedene Dinge.

7. Die erste Bestandsaufnahme

Bevor die erste ernsthafte Produktfunktion beauftragt wird, braucht das System ein reales Bild der Anwendung. Verwenden Sie den Installations- und Anpassungsleitfaden, das Produktentdeckungsgespräch und die Übernahmeregeln. Bei vorhandener Altsoftware ist zusätzlich die Reankerung relevant.

Der Planner prüft, was schon existiert: Code, Datenmodelle, Schnittstellen, Deployment, relevante Betriebsumgebung und die letzten nachweisbaren Anforderungen. Er klärt mit dem CLIENT, welches Verhalten tatsächlich gewünscht wird. Danach hält er die bestätigten Produktanforderungen, technische Grundlage, Eigentumsverhältnisse, nächsten Lieferabschnitte und ihre Abnahmekriterien fest.

Diese Arbeit darf nicht durch das Kopieren von Beispiel-JSON ersetzt werden. Eine zufällige SHA, ein plausibel klingender Kampagnenname oder eine erfundene

Datenbankkennung machen einen Validator vielleicht zufällig zufrieden, aber das Projekt anschließend unzuverlässig. Fehlende Fakten sind offen oder blockiert, bis sie tatsächlich belegt sind.

8. Bestehende Projekte behutsam übernehmen

Eine bestehende Anwendung kann eine wertvolle Historie besitzen, selbst wenn ihre Dokumentation unvollständig ist. Bewahren Sie die vorhandenen Dateien, aktiven Branches, Datenbanken und bereits gültigen Steuerungsregeln zunächst unverändert. Führen Sie eine lesende Bestandsaufnahme aus und vergleichen Sie ihre Ergebnisse mit den gewünschten neuen Kontrollen.

Unterscheiden Sie drei Fälle. Bei einem wirklich neuen Projekt kann das normale Scaffold verwendet werden. Bei einer bestehenden, aber noch nicht geregelten Anwendung ist ebenfalls eine vorsichtige Installation möglich, jedoch nur nach Prüfung des Zielordners. Bei einer bereits geregelten Anwendung ist der Attachment-Preflight erforderlich. Er beurteilt Konflikte und Kompatibilität, ohne durch blindes Überschreiben eine bisherige Verantwortungskette zu zerstören.

Auch ältere Starterversionen sind kein Anlass für ein unkontrolliertes Upgrade. Lesen Sie die Upgrade-Grenzen. Ein öffentlich unterstützter Installationsweg ist nicht automatisch eine geprüfte Migration für jedes Projekt und jeden Host.

9. Zugangsdaten, Anbieter und Feedback

Vergeben Sie nur die Rechte, die eine konkrete Handlung benötigt. Eine lesende Erkundung braucht normalerweise keine Berechtigung zum Löschen, Veröffentlichen, Bezahlen oder Ändern von Produktionsdaten. Bewahren Sie Passwörter, API-Schlüssel und private Zertifikate außerhalb öffentlicher Chatverläufe und des Repositorys auf. Stellen Sie sicher, dass eine Verbindung zu GitHub nicht versehentlich einen fremden Fork oder das Starterrepository anstelle Ihrer Anwendung verändert.

Der Starter unterstützt lokale, datensparsame Architektur-Rückmeldungen. In v1.4 ist ein kleiner Metadatenbericht standardmäßig lokal möglich; eine automatische externe Übermittlung an den Maintainer ist damit nicht eingerichtet. Der Parameter –no-feedback-workflow entfernt die periodische GitHub-Actions-Zeitplanung, deaktiviert aber nicht den lokalen Berichtsgenerator. Wenn Sie keinerlei Berichte wollen, setzen Sie enabled=false in .agentic-sdlc/feedback/ARCHITECTURE_FEEDBACK_CONFIG und prüfen die Datenschutzbeschreibung.

Diese Trennung ist absichtlich fein: Ein geplanter Workflow, ein lokal ausführbares Werkzeug und eine externe Übertragung sind unterschiedliche Handlungen. Die Zustimmung zu einer davon darf nicht als pauschale Freigabe für die anderen interpretiert werden.

10. Fehlerdiagnose und Abschlusskontrolle

Beobachtung	Wahrscheinliche Bedeutung	Sichere Maßnahme
Python wird nicht gefunden	Interpreter fehlt oder Pfad stimmt nicht	Interpreter und Terminal prüfen

Beobachtung	Wahrscheinliche Bedeutung	Sichere Maßnahme
Zielordner bleibt leer	Befehl scheiterte oder falsches Ziel gewählt	Zielparameter und Ausgabe vergleichen
HOLD / UNTRUSTED_CONTEXT	Projektfakten wurden noch nicht bestätigt	Mit lesender Discovery und Adoption fortfahren
jsonschema fehlt	Aktive Python-Umgebung hat keine Validierungsabhängigkeit	Deklarierte Requirements installieren
Vorhandener Steuerungsordner wird geschützt	Der Installer verweigert ein Überschreiben	Geregeltes Projekt zunächst lesend prüfen
Rekonstruktion meldet Widerspruch	Code, Autorität oder Zustand passen nicht zusammen	Ausgaben sichern und Ursache untersuchen
Der Agent sieht das falsche Repository	Arbeitsverzeichnis oder Werkzeugkontext ist falsch	Schreibzugriff anhalten und Pfade prüfen
Kontrolltests sind grün, die App startet nicht	Es wurde nicht das tatsächliche Produkt getestet	Reale Ausführung und Nutzerreise prüfen

Wenn ein Fehler wiederkehrt, klassifizieren Sie ihn, bevor Sie etwas erneut installieren: Umgebung, Berechtigung, falscher Pfad, nicht unterstützte Version, fehlende Abhängigkeit, widersprüchliche aktuelle Referenz oder beschädigter Zustand. Beheben Sie die Ursache und bewahren Sie relevante Belege. Ein frischer Clone oder das Löschen des Steuerungsordners ist keine allgemeine Reparaturmethode.

Sie haben dieses Kapitel abgeschlossen, wenn die richtige Installation anhand ihres konkreten Pfades und Versionsstands nachgewiesen ist, vorhandener Anwendungscode unbeschädigt blieb und das System seine noch unbekannten Projektfakten ehrlich anzeigt. Der nächste Schritt ist nicht sofortige Entwicklung, sondern eine belastbare Übernahme des realen Projektzustands.

Weiter: Kapitel 02, Planner und Adoption.

Quellen und Gültigkeit

- GitHub-Release v1.4.0
- Öffentlicher v1.4-Einstieg
- Versionen und Upgrade
- Fork und Bootstrap
- Übernahmevertrag
- Prüfung bestehender Steuerung
- Metadatenfeedback und Datenschutz
- Scaffold-Quellcode

Die Beispiele sind keine Bestätigung für eine konkrete Installation oder für die reale Ausführung eines anderen Produkts.

02 — Planner und Projektübernahme

> Nach der Installation besitzt das Projekt eine Steuerungsebene, aber noch keinen automatisch gültigen Produktzustand. Dieses Kapitel beschreibt die Entdeckung, Bestätigung und Adoption. Maßgeblich sind die Produktentdeckungsregeln und der Adoptionsvertrag.

In diesem Kapitel

1. Warum der Gesprächsverlauf nicht der Projektzustand ist
2. Der erste lesende Rundgang
3. Produktfragen und technische Fragen trennen
4. Bestätigte Anforderungen erstellen
5. Technische Grundlage und Lieferplan
6. Adoption als überprüfbare Zustandsänderung
7. Der Test mit einem neuen Kontext
8. Den ersten Entwicklungsauftrag abgrenzen
9. Ein tatsächliches Erstgespräch
10. Häufige Fehler und Abnahme des Baselines

1. Warum der Gesprächsverlauf nicht der Projektzustand ist

Nach einer erfolgreichen Installation zeigt ArquitecturaRepo absichtlich einen vorsichtigen Ausgangszustand. Der Starter kennt das neue Produkt noch nicht. Ein in einem Beispielordner vorhandener Product State ist nicht die Beschreibung Ihrer Anwendung. Selbst wenn ein Assistent in einem Gespräch bereits eine perfekte Zusammenfassung der gewünschten Funktionen erzeugt hat, sind diese Aussagen noch nicht automatisch mit aktuellem Code, echten Daten und einer bestätigten Verantwortungskette verbunden.

Ein Planner muss daher eine systematische Bestandsaufnahme durchführen. Er klärt zwei unterschiedliche Bereiche: Welche Produktabsicht wurde vom CLIENT wirklich bestätigt? Und welche technische Situation lässt sich am vorhandenen Projekt nachweisen? Diese beiden Antworten können voneinander abweichen. Ein Auftraggeber kann einen Mitgliederexport wünschen, während die vorhandene Anwendung noch nicht einmal Mitglieder persistent speichert. Umgekehrt kann der Code alte Zahlungsfunktionen enthalten, die in der nächsten Version ausdrücklich nicht mehr verwendet werden sollen.

Die Aufgabe besteht nicht darin, ein möglichst beeindruckendes Dokument zu schreiben. Es geht um die Fähigkeit einer späteren Sitzung, gültige Entscheidungen und technische Tatsachen voneinander zu unterscheiden. Eine früher formulierte technische Hypothese darf durch neue physische Belege korrigiert werden. Eine tatsächlich neue Produktentscheidung muss zur aktuellen Anforderung promoviert werden, ohne die historische Entscheidung unkenntlich zu machen.

Sobald wesentliche Fakten fehlen, kennzeichnet der Planner diese als offen. Er ersetzt sie nicht durch allgemeine Branchenannahmen. Ein sicher wirkender, aber erfundener Datenbankstand ist gefährlicher als ein offen benannter unbekannter

Stand, denn er kann falsche Migrationen oder Datenverluste auslösen. Dasselbe gilt für nicht bestätigte Freigaben, angeblich verfügbare Geheimnisse und vermeintlich „aktuelle“ Branches.

2. Der erste lesende Rundgang

Beginnen Sie mit den tatsächlich vorhandenen Arbeitsoberflächen. Der Planner ermittelt den Repositorypfad, den aktuellen Git-Stand, bestehende Änderungen, vorhandene Tests, Startbefehle, Datenbankkonfiguration, Umgebungsvarianten und etwaige bereits gültige Regelwerke. Falls ein Tool eine andere Arbeitsmappe verwendet, wird diese Abweichung zuerst geklärt.

Eine sinnvolle Inspektionsreihenfolge lautet:

1. Identität der Anwendung und ihr beabsichtigter Nutzerkreis.
2. Aktuelle Dateien und Branches; bekannte ungesicherte Änderungen bleiben erhalten.
3. Bereits vorhandene Anforderungen, Architekturentscheidungen und Produktzustände.
4. Technische Abhängigkeiten, Datenhaltung, externe Dienste und Startpfade.
5. Vorhandene Tests, reale Ausgaben und bekannte Fehlerberichte.
6. Gültige Rollen, Arbeitsfreigaben und Laufzeitbeschränkungen.
7. Unbekannte Fakten mit ihrer Auswirkung auf den nächsten Schritt.

Die Bestandsaufnahme erfolgt zunächst ohne destruktive Befehle und ohne externe Produktänderungen. Ein bestehendes Repository braucht normalerweise keinen neuen Clone. Eine funktionierende Datenbank muss nicht „vorsichtshalber“ neu angelegt werden. Denken Sie beim Arbeiten mit großen Dateien auch an temporäre Caches, Bilder und Artefakte; erhalten Sie nur jene Kopien, die eine konkrete Beweis- oder Wiederherstellungsfunktion besitzen.

Der Product-Discovery-Prompt und die technische Baseline-Regel unterstützen diesen Rundgang. Sie sind Leitplanken für konkrete Belege, nicht ein Formular, das blind mit plausiblem Text gefüllt werden soll.

3. Produktfragen und technische Fragen trennen

Ein Kunde kann verantwortlich entscheiden, wer ein System nutzt, welche Informationen angezeigt werden sollen und was geschieht, wenn ein Vorgang fehlschlägt. Der Kunde muss nicht die Struktur des Datenbankindexes, die Retry-Strategie oder die Gruppierung interner Klassen bestimmen. Wenn die technische Umsetzung die Produktabsicht gefährdet, soll der Planner verständliche Alternativen und Konsequenzen erläutern.

Frage an den CLIENT	Technische Verantwortung
Sollen mehrere Mitarbeiter gleichzeitig Datensätze bearbeiten?	Konkurrenzkontrolle, Transaktionen, Konfliktbehandlung
Was bedeutet „Mitglied deaktivieren“ fachlich?	Persistenzmodell, Beziehungen, Migrationsstrategie
Welche Informationen sind vertraulich?	Verschlüsselung, Zugriffskontrolle, Protokollierung
Muss die Anwendung ohne Internet funktionieren?	Lokale Speicherung, Synchronisierung, Wiederanlauf

Frage an den CLIENT	Technische Verantwortung
Was soll spätestens zum vereinbarten Termin nutzbar sein?	Aufgabenteilung, Testarchitektur, Lieferfolge
Darf die Anwendung echte Zahlungen abwickeln?	Anbieterintegration nur nach gesonderter Autorisierung

Stellen Sie Fragen so, dass eine Antwort die spätere Software verändert. „Soll der Button blau oder grün sein?“ ist sinnvoll, wenn eine visuelle Präferenz tatsächlich wichtig ist. „Welche SQL-Transaktion soll im Controller geöffnet werden?“ ist normalerweise keine Kundensache. Der Auftraggeber sollte sich auf Problem, Nutzer, Verhalten, Vertrauen, Risiken und subjektive Qualität konzentrieren können.

Ein gutes Discovery-Gespräch entdeckt auch Widersprüche. Ein Nutzer fordert vielleicht „alles endgültig löschen“, erwartet aber zugleich eine lückenlose Historie jeder Beitragszahlung. Beide Ziele können nicht ohne zusätzliche Regeln gleichzeitig erfüllt werden. Der Planner macht die Spannung sichtbar und holt eine Entscheidung ein, bevor eine irreversible Datenoperation gebaut wird.

4. Bestätigte Anforderungen erstellen

Die aktuelle Anforderungssammlung muss in normaler Sprache beantworten, was die Software tun soll und wie Erfolg beobachtet wird. Jede wesentliche Behauptung sollte auf eine echte CLIENT-Entscheidung oder eine ausdrücklich bestätigte bestehende Eigenschaft zurückführbar sein. Ein früheres Gespräch kann diese Herkunft belegen, reicht aber nicht als Beweis für den technischen aktuellen Zustand.

Für das Vereinsbeispiel könnte eine erste vorgeschlagene Grundlage lauten:

Dimension	Vorgeschlagene erste Version	Status
Hauptnutzer	Eine Verwaltungskraft am Vereinsrechner	Bestätigung erforderlich
Mitgliedsdaten	Name, Kontakt und Mitgliedsstatus	Bestätigung erforderlich
Beiträge	Beitragspflicht, Zahlungen und Historie	Bestätigung erforderlich
Deaktivierung	Historische Zahlungen bleiben erhalten	Bestätigung erforderlich
Netzwerk	Zunächst lokal, nicht öffentlich im Internet	Bestätigung erforderlich
Oberfläche	Lesbare Masken und verständliche Fehlermeldungen	Bestätigung erforderlich
Wiederherstellung	Backup-Intervall und maximaler Datenverlust	Offen
Ausschlüsse	Keine Onlinezahlung und keine mobile App	Bestätigung erforderlich

Diese Tabelle ist absichtlich ein Vorschlag, kein behaupteter Istzustand. Der CLIENT korrigiert sie und bestätigt die Produktanforderungen. Danach kann die technische

Rolle daraus konkrete Prüfbedingungen ableiten. Wird später eine Entscheidung ersetzt, muss die gültige Fassung klar erkennbar bleiben.

Eine gute Anforderung beschreibt Grenzfälle. Wer darf Mitglieder deaktivieren? Was passiert bei einem doppelten Eintrag? Darf ein Mitglied ohne Telefonnummer angelegt werden? Bleibt eine Zahlung nach einer Namensänderung auffindbar? Wie wird ein Fehleintrag korrigiert? Wenn die Antwort funktional wichtig ist, soll sie nicht dem Zufall einer Implementierung überlassen werden.

Schreiben Sie aber nicht jede winzige UI-Operation in einen eigenen strategischen Vertrag. Ziel ist eine zusammenhängende Fähigkeit, die sich vom Nutzerproblem bis zum Test verfolgen lässt. Zu viele getrennte Mikrotickets können den Zusammenhang zerstören und das Projekt stärker verlangsamen als fehlende Dokumentation.

5. Technische Grundlage und Lieferplan

Nach der Produktbestätigung bestimmt der Planner die technische Baseline. Sie umfasst mindestens die relevanten Laufzeitversionen, Programmiersprache oder Stack, Datenbank, Sicherheits- und Netzwerkgrenzen, Start- und Bereitstellungspfad sowie die Möglichkeiten zur Sicherung und Wiederherstellung. Diese Angaben müssen zum tatsächlich vorhandenen System passen, nicht nur zu einem bevorzugten Architekturentwurf.

Eine Technologieauswahl ist eine begründete ingenieurtechnische Entscheidung. Für eine lokale Anwendung kann eine eingebettete Datenbank geeignet sein; für mehrere gleichzeitige Nutzer können andere Anforderungen entstehen. Der Planner wägt Fehlerfolgen, Wartbarkeit, Personal, Betrieb und Lieferfrist ab und hält die Entscheidung mit ihrer Gültigkeit fest. Nicht jede erfolgreiche Demonstration beweist, dass dieselbe Architektur später für Internetbetrieb oder hohen Durchsatz geeignet ist.

Der Lieferplan verbindet das bestätigte Ziel mit erreichbaren Terminen und Fähigkeiten. Er enthält nicht zwingend einen riesigen Gantt-Plan. Ein sinnvoller erster Abschnitt kann Mitglieder anlegen und wiederfinden, ein zweiter Beiträge erfassen, ein dritter Auswertungen anzeigen. Jede Fähigkeit sollte ein beobachtbares Ergebnis und passende Akzeptanzkriterien besitzen.

Priorisieren Sie nach Nutzwert, Voraussetzungen und Risiko. Eine Datenbankmigration mit Verlustgefahr braucht zusätzliche Kontrollen. Eine unsichere externe Schnittstelle darf nicht nur deshalb zuerst kommen, weil ein Agent sie besonders leicht generieren kann. Gleichzeitig sollte der Planner keine theoretische Vollständigkeit verlangen, die das erste echte Nutzerfeedback auf unbestimmte Zeit verschiebt.

Der Baseline-Guard und die zugehörigen Schemata helfen beim Erkennen formaler Lücken. Ein Schema kann prüfen, ob ein Feld vorhanden ist. Es kann nicht allein nachweisen, dass dessen Wert wahr oder mit der Kundenabsicht konsistent ist. Deshalb gehören Quellenprüfung und späterer Frischkontext-Test dazu.

6. Adoption als überprüfbare Zustandsänderung

Adoption bedeutet, die zuvor rekonstruierten und bestätigten Fakten in eine konsistente Steuerungsebene zu überführen. Dies ist etwas anderes als die Installation von .agentic-sdlc/. Bei der Adoption werden unter anderem gültige Anforderungen, Product State, technische Baseline, Zuständigkeiten, Quellstand und gegebenenfalls ein Lieferplan miteinander verbunden.

Eine verantwortliche Adoption benutzt eine nachvollziehbare Spezifikation und einen Preflight. Der Preflight erkennt Konflikte zwischen erwarteten und tatsächlich vorhandenen Dateien, IDs, Zuständen und Referenzen. Erst wenn die Voraussetzungen passen, darf die Übernahme der Daten erfolgen. Weder ein grüner JSON-Schema-Test noch die Existenz einer Datei mit CURRENT im Namen reicht allein als Nachweis.

Die Werkzeuge discover_project.py, apply_adoption.py und reconstruct_context.py repräsentieren verschiedene Phasen. Discovery beobachtet. Adoption schreibt nach Freigabe und Preflight. Reconstruction versucht, den aktuellen Sinn wiederherzustellen. Vertauschen Sie diese Schritte nicht und verwenden Sie die tatsächlichen, dokumentierten Parameter des installierten Starters.

Erscheint nach einer Adoption ein Widerspruch, wird die betroffene materielle Ausführung angehalten. Der zuständige technische Bearbeiter untersucht den exakten Grund: eine veraltete SHA, fehlende Datenbankmigration, widersprüchliche Rollen, nicht mehr gültige Anforderung oder fehlerhafte Referenz. Unabhängige lesende Analyse darf weiterlaufen. Man sollte nicht einen zweiten, konkurrierenden Product State erzeugen, nur um die Meldung verschwinden zu lassen.

7. Der Test mit einem neuen Kontext

Die wichtigste Prüfung ist ein echter Kontextwechsel. Eine Person oder ein Agent, die oder der die bisherigen Gespräche nicht kennt, liest die vorgesehenen Repository-Oberflächen und beantwortet:

- Welches Produkt soll für wen entstehen?
- Welche Anforderungen hat der CLIENT ausdrücklich bestätigt?
- Welche Version des Codes und welche relevante Datenbank sind aktuell?
- Welche technische Grundlage gilt?
- Wer darf die nächste Entscheidung treffen?
- Welche Kampagne ist gültig und was genau erlaubt sie?
- Welche Fähigkeit wurde nachgewiesen und welche Gates bleiben offen?
- Welche Handlung ist als nächste erlaubt, und welche wäre unzulässig?

Wenn diese Fragen nur beantwortet werden können, indem der ursprüngliche Auftraggeber eine Stunde lang den Chatverlauf nacherzählt, ist die Übernahme noch nicht gelungen. Der Fehler kann eine fehlende Quelle, eine widersprüchliche Referenz oder eine zu verstreute Dokumentation sein. Beheben Sie das ursächlich; ein neuer zusammenfassender Chattext ohne Quellenbindung genügt nicht.

Ein Frischkontext-Test muss auch negatives Wissen erhalten. „Der Zahlungsdienst ist noch nicht autorisiert“, „Die Datenbank ist möglicherweise älter als der Code“ oder „Die Oberfläche wurde noch nicht von einem echten Nutzer geprüft“ sind wertvolle Zustandsinformationen. Sie verhindern, dass ein späterer Bearbeiter aus Schweigen falsche Freigaben ableitet.

8. Den ersten Entwicklungsauftrag abgrenzen

Erst nach bestätigter Produktgrundlage und nachvollziehbarer Adoption wird ein materieller Entwicklungsauftrag entworfen. Der Planner betrachtet das Gesamtsystem, die Hauptabhängigkeiten, relevante Daten, Fehlerfolgen, Ressourcen und Akzeptanz. Er führt eine strategische Hauptprüfung und eine gesonderte adversariale Prüfung durch, wenn die Arbeit einen entsprechenden Charter benötigt.

Der Charter beschreibt die gewünschte Fähigkeit, ihre Grenzen, Zuständigkeit, Schnittstellen, Abnahmekriterien und Bedingungen, unter denen die Vereinbarung ungültig wird. Er ist kein minutiöser Stundenplan für jede Codezeile. Nach der Autorisierung übernimmt I3 die tatsächliche Zerlegung, Implementierung, Fehlersuche, Integration und technische Prüfung.

Angenommen, die erste Fähigkeit lautet „Neue Mitglieder mit Pflichtfeldprüfung registrieren und später wiederfinden“. Dazu gehören nicht nur ein Formular und ein grüner Unit-Test. Auch Speicherung, doppelte Einträge, Berechtigungen, Fehlerverhalten und die tatsächliche Bedienbarkeit müssen nach dem vereinbarten Umfang betrachtet werden. Die konkrete technische Aufgabenteilung bleibt bei I3.

Ein fehlgeschlagener Regressionstest führt zur Korrektur innerhalb derselben Kampagne. Eine neue CLIENT-Anforderung wie „Mitglieder müssen sich nun selbst im Internet registrieren“ verändert dagegen möglicherweise Sicherheitsmodell und Scope. Dann wird die strategische Autorität erneut benötigt. Diese Unterscheidung verhindert gleichzeitig unkontrollierte Scope-Ausweitung und endlose Planungsrunden.

9. Ein tatsächliches Erstgespräch

Der CLIENT erklärt: „Ich brauche eine kleine Verwaltungssoftware für unseren Sportverein. Sie soll Mitglieder, Beiträge und Aktivitäten auf einem lokalen Rechner verwalten. Die erste Version muss einfach sein; Geld für Onlineanbieter ist nicht vorgesehen.“

Der Planner antwortet zunächst mit einer Rückspiegelung der beabsichtigten Nutzerreise. Er fragt, wer Einträge bearbeiten darf, was bei Deaktivierung passiert, ob Beitragshistorie unverändert bleiben muss und welche Frist den größten Nutzwert bestimmt. Er prüft unabhängig vom Gespräch, welche Dateien und Datenstrukturen im Repository existieren.

Danach legt er eine kompakte Anforderungstabelle und eine technische Ausgangsbeschreibung vor. Der CLIENT bestätigt den fachlichen Teil und korrigiert gegebenenfalls die offenen Punkte. Die technische Rolle entscheidet, wie die Daten verlässlich gespeichert, getestet und gesichert werden. Der Planner ordnet die ersten lieferbaren Fähigkeiten und legt eine nachvollziehbare Baseline an.

Eine neue Sitzung wird anschließend gebeten, dieselbe Ausgangslage aus dem Repository zu rekonstruieren. Nur wenn die wesentlichen Aussagen ohne alte Chatnachrichten reproduzierbar sind, ist die gewünschte Kontinuität belegt. Der erste Charter kann dann so formuliert werden, dass ein Implementierungsteam ihn tatsächlich ausführen und prüfen kann.

10. Häufige Fehler und Abnahme des Baselines

Ein typischer Fehler ist es, den Kunden durch technische Rückfragen zum inoffiziellen Architekten zu machen. Ein anderer ist, fehlende fachliche Entscheidungen durch vermeintlich sinnvolle Defaults zu ersetzen. Beides verschiebt Verantwortung falsch. Ein dritter Fehler ist eine zu große Anfangsplanung, die zwar umfangreiche Dokumente erzeugt, aber keine überschaubare erste Fähigkeit sichtbar macht.

Die Baseline ist ausreichend, wenn die Produktabsicht bestätigt, die technische Ausgangslage physisch verifiziert, die Zuständigkeiten klar und die nächste zulässige Aktion rekonstruierbar ist. Nicht jede denkbare künftige Entscheidung muss im Voraus getroffen werden. Materiale Unsicherheit wird gekennzeichnet und dort

behandelt, wo sie tatsächlich eine Handlung beeinflusst.

Prüfen Sie abschließend gegen die Kernregeln des Planners, den Adoptionsvertrag und das Autoritätsmodell. Diese Quellen helfen, produktive Eigenständigkeit und reservierte Entscheidungen sauber auseinanderzuhalten.

Weiter: Kapitel 03, täglicher Betrieb.

Quellen und Gültigkeit

- Produktentdeckung und technische Baseline
- Übernahme/Adoption
- Planner-Kernel
- Produktentdeckungsgespräch
- Discover-Tool
- Adoptionswerkzeug
- Frischkontextrekonstruktion

Die dargestellten Vereinsanforderungen sind Vorschläge für ein fiktives Beispiel, keine bestätigte Realität eines fremden Produkts.

03 — Täglicher Betrieb und kontrollierte Umsetzung

> Dieses Kapitel beschreibt den normalen Arbeitszyklus nach bestätigter Adoption und einem autorisierten Entwicklungsauftrag. Die Regeln beruhen auf dem Execution Team OS, der Kampagnenvereinbarung und der Rekonstruktion aktueller Zustände.

Kapitelübersicht

1. Einen Arbeitstag ohne Neustart beginnen
2. Von einer Produktabsicht zur überprüfbaren Fähigkeit
3. Die operative Verantwortung von I3
4. Externe und reservierte Freigaben
5. Ergebnisse eines Ausführenden kritisch lesen
6. Git, Branches und technische Kontinuität
7. Die tägliche Fünffragenkontrolle
8. Ein vollständiger Tagesablauf
9. Ein klar begrenzter Arbeitsauftrag
10. Zustände und Probleme im Controller
11. Bericht, Prüfung und Rückroute
12. Die Sitzung rekonstruierbar beenden

1. Einen Arbeitstag ohne Neustart beginnen

Ein zuverlässiger Tagesbeginn besteht nicht darin, alle alten Gesprächsnotizen neu zu lesen. Der Bearbeiter öffnet das tatsächlich zuständige Repository und rekonstruiert die aktuellen Fakten. Welcher Branch und welche Arbeitsspur gelten? Welche Änderungen sind ungesichert? Gibt es einen aktiven Charter, eine eindeutige Eigentümerschaft und einen konkreten nächsten Arbeitszustand? Welche früheren Testergebnisse sind noch an den aktuellen Code gebunden?

Diese Prüfung erfolgt möglichst deltaorientiert. Wenn bereits ein vertrauenswürdiger Checkout existiert, muss nicht jedes Mal das ganze Repository neu geklont werden. Ein gezielter Abruf des relevanten Branches, die Sichtung neuer Commits und die Kontrolle der veränderten Dateien reichen oft. Vermeiden Sie wahllose neue Worktrees und dauerhafte Duplikate großer Artefakte.

Bei einem echten Kontextwechsel darf der neue Ausführende nicht annehmen, dass seine eigene Erinnerung aktuell ist. Ein alter Handoff kann eine inzwischen überholte Kampagne enthalten. Umgekehrt kann eine neue Chatnachricht eine Produktentscheidung ausdrücken, die noch nicht im Product State dokumentiert wurde. Technische Wahrheit und Produktabsicht müssen sorgfältig auseinandergehalten werden.

Eine einfache Tagesnotiz kann so lauten: „Gültiger Entwicklungsauftrag: Mitgliedersuche; aktueller Kandidat: konkret benannter Git-Stand; offenes Gate: Datenbankmigration auf realistischem Datensatz; nächste erlaubte Aktion: Test und in-scope Korrektur;

keine Berechtigung zur öffentlichen Veröffentlichung.“ Diese fünf Sätze sind nützlicher als eine unüberprüfbare Zusammenfassung von hundert vergangenen Nachrichten.

2. Von einer Produktabsicht zur überprüfbaren Fähigkeit

Der CLIENT sagt vielleicht: „Ich möchte Mitglieder schnell wiederfinden können.“ Der Planner untersucht zunächst, welche Suche das Arbeitsproblem löst. Geht es um vollständige Namen, Teilstrings, Mitgliedsnummern, Statusfilter oder Duplikate? Wie groß ist die Datenmenge und wer darf die Ergebnisse sehen? Welche Antwortzeit fühlt sich im vorgesehenen Kontext ausreichend an?

Eine gute Fähigkeit ist ein Ende-zu-Ende-Ergebnis aus Sicht des Nutzers. „Suchservice implementieren“ ist eine mögliche technische Aufgabe, aber noch keine vollständige Produktfähigkeit. Ein nutzbarer Suchablauf umfasst Eingabe, Berechtigungen, Datenabfrage, Rückmeldung bei keinen Treffern, Fehlerbehandlung und tatsächliche Anzeige. Der Charter beschreibt die Grenzen und Nachweise dieser zusammenhängenden Wirkung.

Ein technisch kleiner Auftrag darf absichtlich eng gefasst werden. Wenn zuerst nur die sichere Suche nach Mitgliedsnummern benötigt wird, muss nicht sofort eine universelle Suchmaschine entwickelt werden. Der Umfang folgt der bestätigten Produktabsicht und den Voraussetzungen. Ein gutes Framework verhindert Scope-Ausweitung, ohne jede Entwicklung in winzige Planungskampagnen zu zerlegen.

Das Charter-Standarddokument trennt strategische Abnahmebedingungen von taktischen Einzelarbeiten. Die genaue Implementierungsfolge wird erst innerhalb der autorisierten Arbeit entschieden. Das verschafft der Ausführung die notwendige technische Beweglichkeit und lässt den CLIENT bei den produktrelevanten Fragen.

3. Die operative Verantwortung von I3

Nach einer gültigen Freigabe übernimmt der Execution Controller die operative Arbeit. Er entscheidet, welche Dateien und Schnittstellen untersucht werden, wie Fachkräfte eingesetzt werden, welche Tests sinnvoll sind und wie ein Fehler innerhalb des festgelegten Bereichs korrigiert wird. Der Planner muss diese Entscheidungen nicht täglich freigeben.

Zum Arbeitsbereich von I3 gehören Implementierung, Debugging, Integration, interne Refaktorierungen, nötige Migrationen innerhalb des Charters, Testwiederholungen, Wiederaufnahme nach Abstürzen und fachlich begrenzte unabhängige Prüfungen. I3 hält den aktuellen Stand und die Abweichungen vom geplanten Ergebnis nachvollziehbar. Ein Reviewer darf einen Kandidaten wegen eines konkreten Fehlers zurückweisen; die Korrektur bleibt normalerweise in derselben Kampagne.

Angenommen, die Mitgliedersuche liefert bei Umlauten falsche Ergebnisse. Das ist zunächst ein gewöhnlicher Implementierungsfehler. Die zuständige Ausführung untersucht Normalisierung, Datenbankkollation und Testfälle, korrigiert den Code und wiederholt die relevanten Prüfungen. Es wäre ineffizient, hierfür einen neuen strategischen PASS, eine neue Kampagne oder eine neue Kundenbesprechung zu verlangen.

Tritt dieselbe Fehlklasse wiederholt auf, muss die Antwort systemisch werden. Nach mehreren Rückläufen hilft es wenig, nur den nächsten offensichtlichen Testfall zu patchen. I3 untersucht den betroffenen Teilbereich vollständig, überprüft die

Architektur und veranlasst eine echte unabhängige Prüfung, bevor erneut eine strategische Endmeldung abgegeben wird.

Die Grenze ist erreicht, wenn der bestehende Charter durch neue Produktabsicht, unerwartete wesentliche Sicherheitsfolgen oder eine grundlegende Architekturänderung ungültig wird. Dann dokumentiert I3 den Befund und routet die strategische Entscheidung an den richtigen Eigentümer. Gewöhnliche Bugs sind keine solche Ungültigkeit.

4. Externe und reservierte Freigaben

Ein ausführender Agent kann nicht allein deshalb auf einen Dienst zugreifen, weil Anmeldedaten vorhanden sind. Er muss wissen, ob die konkrete Handlung vom CLIENT und den technischen Zuständigkeiten erlaubt ist. Lesende lokale Untersuchung, Testen in einer isolierten Umgebung und das Erstellen eines Entwurfs unterscheiden sich von Veröffentlichungen, Geldausgaben, echten Kundennachrichten und Änderungen an Produktionsdaten.

Behalten Sie für sensible Operationen Ziel, Umfang und zulässige Kosten fest. Ein allgemeiner Satz wie „Mach das Projekt fertig“ autorisiert keine unbegrenzten kostenpflichtigen API-Aufrufe und keine Veröffentlichung personenbezogener Daten. Ebenso ist die Anwesenheit eines Tools keine Genehmigung, es außerhalb der vorgesehenen Umgebung zu verwenden.

Eine vernünftige Freigabe lautet zum Beispiel: „Du darfst in der lokalen Testdatenbank die Migration ausführen, nachdem das Backup geprüft wurde; Produktionsdaten bleiben unverändert.“ Eine solche Aussage lässt konkrete Kontrollen zu. Bei einem unbekannten Produktionszustand bleibt die abhängige Handlung auf HOLD, während lesende Diagnose weiterlaufen darf.

Bei dritten Inhalten müssen Urheberrechte, Nutzungsbedingungen und tatsächliche Lizenzgrenzen berücksichtigt werden. Verlangen Sie eine angemessene Nutzungsberechtigung, aber stellen Sie nicht jede harmlose lokale technische Prüfung durch endlose rein formale Dokumentationsforderungen still. Wichtig ist die reale Rechts- und Sicherheitslage.

5. Ergebnisse eines Ausführenden kritisch lesen

Ein guter Bericht benennt den tatsächlich veränderten Code, die Dateien, den überprüften Kandidaten, seine Testumgebung und das Ergebnis jedes relevanten Gates. Ein schlechter Bericht behauptet nur „fertig“ und zählt Commits, Dateien oder Testfälle. Die reine Zahl bestandener Tests sagt nichts darüber, ob die richtige Nutzerfunktion gebaut wurde.

Lesen Sie besonders auf drei Stellen:

- **Behauptung:** Was wird ausdrücklich als funktionierend bezeichnet?
- **Zeuge:** Welcher konkrete Code, Datenbankstand oder Artefakt beweist das?
- **Grenze:** Welche Umgebung, Eingabe, Nutzerprüfung oder Freigabe wurde nicht abgedeckt?

Wenn die Quelle seit dem Test verändert wurde, ist der frühere Test nicht automatisch aktuell. Ein Prüfer muss den exakten Kandidaten nachweisen, gegen den er geprüft hat. Ein neuer Commit kann einen bekannten Fehler beheben und zugleich einen anderen Fehler eingeführt haben. Daher benötigen materielle Änderungen eine

passende erneute Prüfung.

Ein visuell überzeugendes Video kann die Bedienbarkeit zeigen, aber keine Datenbanksicherheit beweisen. Umgekehrt kann ein ausführlicher Unit-Test nicht belegen, dass die Oberfläche tatsächlich lesbar oder für den vorgesehenen Nutzer verständlich ist. Jede Aussage braucht den dazu passenden Nachweis.

6. Git, Branches und technische Kontinuität

Git speichert Quellhistorie. Es ist kein automatisches Register dafür, welcher Mensch oder Agent die tatsächliche Arbeit ausgeführt hat. Ein Commit zeigt eine Änderung und ihren Autorbeleg, aber nicht zwangsläufig die vollständige Rollenidentität, Freigabe oder den aktuellen Zustand der Produktdatenbank.

Für mehrere gleichzeitige Aufgaben beurteilen Sie, welche Dateien und Ressourcen tatsächlich betroffen sind. Zwei Aufgaben in verschiedenen Produktbereichen können denselben Migrationsordner oder dieselbe Konfigurationsdatei verändern und sich damit stören. Zwei Aufgaben im selben Repository können hingegen unabhängig sein, wenn ihre Oberflächen wirklich getrennt sind. Planen Sie Konkurrenz nach Interferenz, nicht nur nach Projektname.

Bewahren Sie ungesicherte Änderungen. Benutzen Sie weder einen erzwungenen Reset noch das Löschen des gesamten Arbeitsverzeichnisses als Standardlösung für einen unklaren Zustand. Ein temporärer Worktree kann für echte Parallelität sinnvoll sein; entfernen Sie ihn kontrolliert, wenn er keine aktive Autorität oder Beweisfunktion mehr besitzt.

Die aktuellen technischen Referenzen werden zusammen mit ihrem Gültigkeitsbereich gehalten. Eine Änderung im Branch macht nicht automatisch jede frühere Annahme über Datenbankzustand oder Artefakte falsch. Umgekehrt macht ein unveränderter Commit keine inzwischen geänderte externe API oder Datenbank automatisch aktuell.

7. Die tägliche Fünffragenkontrolle

Am Ende einer Arbeitseinheit lohnt sich eine kurze, wiederholbare Prüfung. Erstens: Welche nutzbare Fähigkeit wurde tatsächlich verbessert? Zweitens: Welcher konkrete Kandidat und welche Belege liegen vor? Drittens: Welche Risiken oder offenen Gates bleiben? Viertens: Welche Zuständigkeit hat die nächste Entscheidung? Fünftens: Könnte eine neue Sitzung morgen diese Antworten ohne Gesprächsarchäologie selbst rekonstruieren?

Wenn die Antworten ausschließlich „zehn Tests mehr“, „drei neue Agenten“ oder „noch ein großer Bericht“ lauten, fehlt möglicherweise die Verbindung zum Produkt. Das ist kein Vorwurf an den Arbeitsaufwand. Es ist ein Signal, die nächste technische Handlung wieder an überprüfbare Fähigkeiten und Risiken zu binden.

Die Tageskontrolle soll kompakt bleiben. Sie ersetzt nicht die genaueren Kampagnen- und Gate-Nachweise, aber sie verhindert, dass ein scheinbar aktives Projekt seinen eigentlichen Zweck aus den Augen verliert. Sie darf auch das Ergebnis „kein Fortschritt wegen fehlender autorisierter Datenbank“ zeigen, solange der Blocker präzise und der nächste sichere Schritt klar ist.

8. Ein vollständiger Tagesablauf

Am Morgen öffnet die technische Rolle die Vereinsanwendung und bestätigt den letzten freigegebenen Charter. Die Aufgabe ist eine robuste Suchfunktion für Mitglieder. Der aktuelle Kandidat stammt von einem konkreten Git-Stand, und eine Testdatenbank enthält synthetische Mitgliederdaten. Die echte Vereinsdatenbank wird nicht berührt.

Zunächst läuft eine kleine Regression der vorhandenen Mitgliedsverwaltung. Dann implementiert der Ausführende einen zusätzlichen Suchfilter. Ein Test zeigt, dass deaktivierte Mitglieder fälschlich unsichtbar werden, obwohl die Anforderung ihren historischen Zugriff erlaubt. I3 prüft den betroffenen Filter, korrigiert ihn und ergänzt einen Regressionstest.

Anschließend überprüft ein unabhängiger Blick die gesamte Suche unter unterschiedlichen Datenzuständen: kein Treffer, viele Treffer, Namensgleichheit, ungültige Eingaben und unberechtigte Zugriffe. Technische Tests werden um eine echte UI-Demonstration ergänzt. Wenn die Bedienelemente schwer verständlich sind, wird der Unterschied zwischen technischer Korrektheit und wahrgenommener Qualität sichtbar.

Am Abend wird der Kandidat mit dem exakten Quellstand, den Ergebnissen und den offenen Grenzen dokumentiert. Der Planner erhält einen konsolidierten strategischen Bericht, sobald eine wirkliche Abnahmeentscheidung ansteht. Ein weiteres kleines Testversagen am nächsten Tag gehört weiterhin zur technischen Ausführung und löst nicht automatisch einen neuen Charter aus.

9. Ein klar begrenzter Arbeitsauftrag

Eine gute Übergabe an I3 enthält Ziel, aktuelle Quellen, erlaubte Grenzen, notwendige Checks und die Definition eines vorlegbaren Ergebnisses. Eine kompakte Fassung könnte lauten:

> Ziel: Die bestätigte Mitgliedersuche end-to-end verfügbar machen. Ausgangspunkt: aktueller Anwendungsbranch und synthetische Testdaten. Grenzen: keine Produktionsdatenänderungen, kein neuer externer Anbieter, keine Erweiterung des Mitgliedsmodells ohne Rückfrage. Gates: Unit, Integration und reale UI-Prüfung getrennt nachweisen. Abschluss: exakten Kandidaten, Testergebnisse, Gegenbelege und offene Risiken melden.

Ein solcher Auftrag überlässt technische Detailarbeit dem dafür zuständigen Controller. Er schützt zugleich die Produktabsicht und die externen Befugnisse des CLIENT. Zu lange Übergaben mit vollständiger Chatgeschichte schaffen eher Widersprüche, wenn dieselbe Information im Repository bereits aktuell vorhanden ist.

Nach der Übergabe bleibt I3 verantwortlich, bis ein sinnvoller terminaler Kandidat vorliegt oder eine echte Charter-Ungültigkeit festgestellt wird. Das Framework soll nicht bei jedem Debugging-Schritt die Verantwortung zurück an den Auftraggeber werfen.

10. Zustände und Probleme im Controller

Der optionale lokale campaignctl.py kennt Zustände für die von ihm verwalteten Kampagnen. Eine Zustandsänderung in diesem Werkzeug ist nur ein Nachweis über

seine eigene kontrollierte Oberfläche. Sie beweist weder, dass ein externer Coding-Agent tatsächlich ausgeführt wurde, noch dass Produktdaten korrekt migriert oder die Nutzeroberfläche abgenommen wurden.

Wenn der Controller einen offenen Vorgang meldet, prüfen Sie zuerst die zugehörige Identität, den letzten gültigen Checkpoint und die vorhandenen Quellen. Ein Neustart des Programms darf einen bereits verbuchten externen Effekt nicht wiederholen. Für irreversible Aktionen sind idempotente Quittungen und klare Ownership wichtig.

Besonders gefährlich sind stillschweigende Reparaturen: eine ungültige Datenbankreferenz durch eine beliebige andere ersetzen, einen fehlenden Reviewer durch ein eigenes PASS markieren oder einen alten Charter nur durch Umbenennung als aktuell deklarieren. Die korrekte Reaktion ist eine kausale Diagnose mit nachvollziehbarer Wiederherstellung.

11. Bericht, Prüfung und Rückroute

Der Bericht am Ende einer Kampagne sollte die gebaute Fähigkeit, den exakten Kandidaten, die gemessenen Ergebnisse und die offenen Probleme benennen. Ein unabhängiger Review muss gegen denselben Kandidaten erfolgen. Wenn der Reviewer wesentliche Fehler findet, werden diese im bestehenden Auftrag korrigiert. Eine erneute strategische Planung ist nur bei veränderter Absicht oder materiell neuer Architektur- beziehungsweise Risikolage erforderlich.

Eine strategische Annahme beurteilt andere Fragen als ein Test. Hat die Umsetzung tatsächlich das vereinbarte Produktverhalten geliefert? Sind die Schnittstellen und abhängigen Fähigkeiten konsistent? Wurde ein neues relevantes Risiko geschaffen? Sind die richtigen menschlichen Entscheidungen eingeholt worden?

Für die Qualität der Oberfläche muss der CLIENT oder eine geeignete reale Nutzerprüfung einbezogen werden, wenn das Produkt dies verlangt. I3 kann die Oberfläche technisch liefern, darf seine eigene ästhetische Einschätzung aber nicht als endgültige Kundenfreigabe ausgeben.

12. Die Sitzung rekonstruierbar beenden

Bevor Sie schließen, sichern Sie die gültigen aktuellen Artefaktreferenzen, den Stand der Arbeitsfreigabe, die Tests und das nächste zulässige Vorgehen auf den dafür vorgesehenen kanonischen Oberflächen. Halten Sie einen nachweislich alten Bericht nicht neben einer neuen „Wahrheit“ als gleichrangige aktuelle Quelle. Markieren Sie Supersessionen und offene Unsicherheiten.

Legen Sie keinen riesigen neuen Handoff an, der dieselben Regeln mehrfach kopiert. Dauerhafte Zustandsinformationen und stabile Richtlinien benötigen unterschiedliche Eigentümer und Speicherorte. Eine kurze Navigation zur maßgeblichen Quelle ist oft wertvoller als eine weitere vollständige Abschrift.

Der Abschlusstest ist schlicht: Ein anderer Bearbeiter öffnet morgen das Repository, prüft die aktuelle Autorität und kann sicher fortfahren, ohne Ihre heutige Unterhaltung zu kennen. Genau diese Kontinuität rechtfertigt den organisatorischen Aufwand.

Weiter: Kapitel 04, Änderungen anfordern.

Quellen und Gültigkeit

- Ausführungsteam und Verantwortungen
- Kampagnen-Charter
- Aktualität und Rekonstruktion
- Quell- und Arbeitsbereichseffizienz
- Review- und Terminalintegrität
- Lokaler Kampagnencontroller

Das Kapitel beschreibt den belegbaren Kontrollzyklus; es behauptet keine automatische Ausführung externer Programmierdienste.

04 — Produktänderungen richtig anfordern

> Der Auftraggeber soll eine gewünschte Wirkung beschreiben können, ohne die technische Umsetzung selbst zu verwalten. Dieses Kapitel zeigt, wie Änderungen klar, überprüfbar und mit nachvollziehbarer Priorität formuliert werden. Verbindliche Grenzen stehen im CLIENT-Boundary-Protokoll und im Anforderungsprozess.

Kapitelübersicht

1. Mit dem Ergebnis beginnen
2. Ein kompakter Änderungsauftrag
3. Welche Entscheidungen beim CLIENT bleiben
4. Änderungen der Meinung ohne Geschichtsverlust
5. Qualitätswünsche ohne Technikvorgabe
6. Prioritäten setzen und den Umfang begrenzen
7. Wann der Planner wirklich zurückfragen muss
8. Fünf gut formulierte Beispiele
9. Neue Produktabsicht oder gewöhnlicher Fehler?
10. Wenn Software funktioniert, aber nichts nützt
11. Eine praktische Gesprächsvorlage

1. Mit dem Ergebnis beginnen

Viele Softwareänderungen werden als technische Anweisungen formuliert: „Baue einen REST-Endpunkt“, „füge einen Index hinzu“ oder „nutze eine andere Datenbank“. Solche Aussagen können richtig sein, wenn der Auftraggeber selbst eine technische Randbedingung setzen will. In gewöhnlicher Produktarbeit sind sie aber oft nur Vermutungen über die eigentliche Lösung.

Beginnen Sie besser mit dem Problem und dem beobachtbaren Ergebnis. Statt „Erstelle einen Cache für Mitglieder“ könnten Sie sagen: „Die Verwaltungskraft muss ein Mitglied bei rund zehntausend Einträgen schnell finden können, ohne die gleiche Suche mehrfach zu starten.“ Die technische Rolle untersucht dann, ob ein Index, eine Datenbankabfrage, eine andere Oberfläche oder eine Kombination den gewünschten Effekt zuverlässig erreicht.

Das vermeidet zwei Fehler. Erstens kann ein vorschnell vorgeschriebener Lösungsweg die falsche Ursache behandeln. Zweitens lädt er den CLIENT mit technischer Verantwortung auf, die er weder übernehmen muss noch übernehmen sollte. Der Auftraggeber behält trotzdem die Kontrolle über Qualität, Budget, Zeitplan und den sichtbaren Erfolg.

Ein guter Änderungswunsch beschreibt mindestens: Wer möchte was erreichen? Welcher konkrete Ablauf funktioniert heute nicht oder soll sich verbessern? Woran erkennt die Person die neue Fähigkeit? Welche bestehenden Regeln dürfen nicht verletzt werden? Gibt es eine harte Frist, Kostenobergrenze oder externe Freigabe?

2. Ein kompakter Änderungsauftrag

Für alltägliche Wünsche reicht häufig ein kurzer Text. Eine mögliche Vorlage:

> Als Verwaltungsmitarbeiterin möchte ich Beiträge eines Mitglieds nach Zeitraum filtern und die Summe sehen. Der Filter muss bei leeren Ergebnissen verständlich reagieren und darf keine Zahlungen verändern. Die erste Version braucht keinen Export. Wichtig ist die korrekte Summe vor einer aufwendigen Gestaltung.

Diese Formulierung ist verständlich, abgrenzbar und testbar. Sie legt weder die SQL-Abfrage noch die Klassenaufteilung fest. Der Planner kann daraus die notwendigen Schnittstellen und Abnahmekriterien ableiten. Die technische Umsetzung bleibt bei I3.

Für eine größere Änderung ergänzen Sie Daten- und Betriebsfolgen: Gibt es personenbezogene Daten? Sind Nutzerrechte betroffen? Werden bestehende Aufzeichnungen umgeschrieben? Muss das Produkt offline funktionieren? Sollen Dritte eine Nachricht erhalten? Ein abgrenzbarer Änderungsauftrag muss solche Risiken früh sichtbar machen.

Unbekannte Entscheidungen dürfen ausdrücklich offen bleiben. Beispielsweise kann der CLIENT sagen: „Ich weiß noch nicht, ob ehemalige Mitglieder im Suchergebnis erscheinen sollen. Prüft die Auswirkungen und fragt mich, bevor ihr die Regel festlegt.“ Das ist eine ehrliche Arbeitsgrundlage. Eine stillschweigende technische Annahme wäre dagegen eine unautorisierte Produktentscheidung.

3. Welche Entscheidungen beim CLIENT bleiben

Der CLIENT entscheidet über Zweck, Zielgruppe, Prioritäten und sichtbares Verhalten. Er darf eine Gestaltung ablehnen, auch wenn sie technisch korrekt ist. Er entscheidet über das gewünschte Nutzererlebnis und die zulässigen externen Handlungen, soweit sie ihm vorbehalten sind. Technische Rollen dürfen erklären, dass eine bestimmte Kombination von Anforderungen teuer, riskant oder unvereinbar ist, aber die Produktabsicht nicht heimlich austauschen.

Die technische Umsetzung entscheidet dagegen normalerweise, wie ein Datenmodell strukturiert, eine Schnittstelle aufgebaut, ein Fehler behandelt oder ein Test organisiert wird. Der Planner verantwortet die technische Strategie und der Controller die operativen Details. Wenn eine technische Wahl den Budgetrahmen oder die Benutzererfahrung stark beeinflusst, werden ihre Folgen erklärt und gegebenenfalls eine neue Produktentscheidung eingeholt.

Ein Beispiel: Der CLIENT möchte eine Sicherung seiner Vereinsdaten. Er muss erklären, wie viel Datenverlust noch akzeptabel wäre und ob externe Speicherung erlaubt ist. Engineering bestimmt anschließend die konkrete Backuptechnik, die Fehlererkennung und den Wiederherstellungstest. Es wäre verkehrt, den CLIENT nach dem passenden Dateisystem-Flush oder Transaktionsisolationsgrad zu fragen.

Eine andere Grenze sind rechtliche und externe Entscheidungen. Dass ein Agent eine Veröffentlichung technisch ausführen könnte, ersetzt nicht die erforderliche Freigabe. Besonders bei Mitgliederdaten, Bildern Dritter, Zahlungsdiensten und produktiven Änderungen muss klar sein, welche Handlung erlaubt ist und wer sie genehmigt hat.

4. Änderungen der Meinung ohne Geschichtsverlust

Produktentwicklung ist ein Lernprozess. Eine früher bestätigte Anforderung darf später geändert werden. Vielleicht war die Vereinsanwendung zunächst nur lokal geplant, und nach einer erfolgreichen Testphase möchte der CLIENT doch mehrere Standorte anbinden. Die neue Entscheidung ist zulässig, aber sie verändert möglicherweise Netzwerk, Zugriffsrechte und Sicherheitsstrategie.

Die neue Anforderung muss mit ihrer Herkunft und ihrem Zeitpunkt zum aktuellen Zustand erhoben werden. Die frühere Entscheidung kann als historische Begründung erhalten bleiben, darf aber nicht mehr als gleichrangige aktuelle Autorität gelten. Der technische Planner beurteilt die Auswirkungen auf Code, Daten, laufende Kampagnen und geplante Lieferungen.

Ein nicht mehr passender Charter muss nicht durch heimliche Scope-Erweiterung gerettet werden. Wenn die Änderung die ursprünglich erlaubte Fähigkeit wesentlich umdefiniert, wird die entsprechende strategische Entscheidung neu getroffen. Betrifft sie nur eine kleine, bereits erlaubte Korrektur, kann I3 die Arbeit ohne neue Kampagne fortsetzen.

Vermeiden Sie das reflexartige Löschen alter Dokumente. Mit einer nachvollziehbaren Supersession lässt sich später verstehen, weshalb eine technische Entscheidung damals richtig war und heute nicht mehr gilt. Genau diese Herkunft schützt das Projekt vor scheinbaren Widersprüchen zwischen Chatverlauf und Repository.

5. Qualitätswünsche ohne Technikvorgabe

Qualität ist nicht auf Unit-Tests begrenzt. Ein Formular kann alle Eingaben korrekt speichern und trotzdem so unübersichtlich sein, dass der Nutzer im Alltag Fehler macht. Der CLIENT darf daher konkrete Wahrnehmungs- und Bedienungsziele nennen: lesbar, schnell erfassbar, klar bei Fehlern, konsistent mit dem übrigen Produkt oder nutzbar auf einem bestimmten Bildschirm.

Für die Vereinsverwaltung könnte der Wunsch lauten: „Neue Mitglieder sollen ohne Anleitung in höchstens wenigen verständlichen Schritten angelegt werden können. Pflichtfelder müssen erkennbar sein. Bei einer doppelten Person soll das System vor dem Anlegen warnen.“ Diese Aussage lässt mehrere technische Entwürfe offen, macht aber das gewünschte Erlebnis prüfbar.

Technische Qualität wird ebenfalls beschrieben, nur von anderen Eigentümern. Dazu gehören Fehlerbehandlung, Tests, Datenintegrität, Sicherheitskontrollen, Wiederherstellung und Wartbarkeit. Die konkrete Gewichtung muss zu den realen Risiken passen. Eine gefährliche Migration braucht andere Nachweise als eine rein kosmetische Farbänderung.

Fragen Sie nach einer sichtbaren Demonstration, wenn die Oberflächenqualität im Mittelpunkt steht. Ein Screenshot zeigt einen Zustand, aber nicht unbedingt den Ablauf. Eine kurze Aufnahme oder ein tatsächlich bedienbarer Kandidat kann mehr belegen. Sichern Sie den konkreten Kandidaten, damit spätere Veränderungen nicht unbemerkt mit dem geprüften Stand verwechselt werden.

6. Prioritäten setzen und den Umfang begrenzen

Ein häufiger Fehler ist, zehn neue Wünsche gleichzeitig als unabhängig dringende Goals zu behandeln. Das erschwert die Planung und erzeugt unklare Schnittstellen. Stellen Sie zuerst fest, welche Änderungen von anderen abhängen und welche zusammen eine für den Nutzer brauchbare Fähigkeit ergeben.

Nehmen wir an, der Verein wünscht neue Mitgliedskategorien, E-Mail-Erinnerungen, Beitragsberichte und eine hübschere Startseite. Wenn Beitragsberichte die Mitgliedskategorien benötigen, ist die Reihenfolge relevant. E-Mail-Erinnerungen können externe Datenschutz- und Versandfreigaben erfordern. Die Startseite kann hingegen getrennt verbessert werden, solange sie keine gemeinsame technische Ressource blockiert.

Eine hilfreiche Prioritätseinteilung ist: notwendig für den ersten echten Nutzerablauf, wichtig für Sicherheit und Korrektheit, unmittelbar wertvoll aber verschiebbar, später oder ausdrücklich außerhalb des Umfangs. Der CLIENT bestimmt den Nutzwert; Engineering legt Abhängigkeiten und sichere Reihenfolge dar.

Eine angemessene erste Version soll früh genug nutzbar sein, um echte Rückmeldungen zu erzeugen. Das bedeutet nicht, Tests oder Datenschutz zu ignorieren. Es bedeutet, die Grenze so zu ziehen, dass innerhalb dieses Bereichs eine ehrliche, vollständig überprüfte Fähigkeit entsteht und nicht bloß ein großes Versprechen.

7. Wann der Planner wirklich zurückfragen muss

Eine Rückfrage ist nötig, wenn eine ungeklärte Produktentscheidung das Ergebnis materiell verändert. Das gilt beispielsweise für die Löschung historischer Zahlungen, die Freigabe einer öffentlichen Benutzerregistrierung oder den Versand echter Mahnungen. Sie kann auch nötig sein, wenn zwei bestätigte Wünsche unvereinbar sind.

Eine Rückfrage ist normalerweise nicht nötig, weil ein Unit-Test fehlschlägt, eine Abhängigkeit aktualisiert werden muss oder der Controller eine interne Klasse anders aufteilen will. Diese Angelegenheiten gehören in die technische Verantwortung, sofern keine neue wesentliche Gefahr oder Produktfolge entsteht.

Formulieren Sie die Frage mit Konsequenzen. Statt „Soll ich Option A oder B wählen?“ ist besser: „Wenn deaktivierte Mitglieder verborgen werden, sieht die Verwaltung frühere Zahlungen nur über den Archivfilter. Wenn sie sichtbar bleiben, ist die Liste länger, aber der Verlauf leichter auffindbar. Welche Arbeitsweise entspricht Ihrem Verein?“

So muss der CLIENT nicht die ganze Architektur verstehen. Er kann die tatsächliche Nutzerentscheidung treffen. Die technische Rolle übernimmt anschließend ihre Übersetzung in Implementierung und Tests.

8. Fünf gut formulierte Beispiele

Beispiel 1 — Suche: „Eine Mitarbeiterin soll Mitglieder nach Namen oder Nummer suchen können. Deaktivierte Personen müssen auf Wunsch mit angezeigt werden. Die Suche darf keine Datensätze verändern.“ Dies legt Zweck und Grenzen fest, nicht den Index.

Beispiel 2 — Beiträge: „Beim Erfassen einer Beitragszahlung muss die Person,

der Betrag und das Datum sichtbar sein. Ein versehentlicher Doppelklick darf nicht zwei Zahlungen anlegen.“ Hier folgt aus der Nutzerabsicht eine wichtige technische Idempotenzanforderung.

Beispiel 3 — Deaktivierung: „Eine deaktivierte Person darf keine neuen Aktivitäten buchen. Frühere Buchungen und Zahlungen bleiben nachvollziehbar.“ Der Planner muss auf mögliche Konflikte zu bestehenden Datenschutz- oder Aufbewahrungsregeln achten.

Beispiel 4 — Export: „Der Vorstand benötigt monatlich eine Zusammenfassung ohne persönliche Kontaktadressen. Der Export soll nur durch berechtigte Personen möglich sein.“ Der Wunsch enthält zugleich eine Sicherheitsgrenze.

Beispiel 5 — Oberfläche: „Die wichtigsten täglichen Aktionen sollen ohne Fachbegriffe erkennbar sein. Fehlermeldungen müssen erklären, was der Nutzer korrigieren kann.“ Das ist eine Qualitätserwartung, keine CSS-Anweisung.

Zu jedem Beispiel kann Engineering realistische Abnahmekriterien ausarbeiten. Dazu gehören positive Abläufe, Grenzfälle, Auswirkungen auf vorhandene Daten und die passenden getrennten Gates. Der CLIENT muss nicht die Testautomatisierung selbst programmieren.

9. Neue Produktabsicht oder gewöhnlicher Fehler?

Die Unterscheidung ist einfach, aber folgenreich. Ein Fehler liegt vor, wenn das bestehende bestätigte Verhalten falsch umgesetzt wurde. Ein neuer Wunsch verändert dieses Verhalten oder erweitert seinen Geltungsbereich. Im ersten Fall sollte I3 innerhalb der bereits autorisierten Arbeit korrigieren. Im zweiten Fall kann eine neue Entscheidung nötig sein.

Beispiel Fehler: Eine Suche soll laut bestätigtem Charter auch Umlaute finden, tut es aber nicht. Die Implementierung muss repariert werden. Beispiel neue Produktabsicht: Die Suche soll nun zusätzlich externe Vereinsdatenbanken abfragen. Das bringt neue Berechtigungen, Schnittstellen und möglicherweise Datenschutzfolgen mit sich.

Ein wiederkehrender Fehler wird nicht allein durch häufiges Auftreten strategisch. Er kann auf eine systemische technische Ursache hinweisen, die I3 mit umfassendem Hardening und unabhängiger Prüfung behandelt. Erst eine wirkliche Veränderung der zulässigen Grenzen oder eine neue wesentliche Gefahr führt zu einer strategischen Neubeurteilung.

Verantwortlichkeiten dürfen nicht durch Berichtsformate verschwimmen. Wenn der Controller sagt „Ich brauche eine neue Kampagne, weil der Reviewer zwei Defekte gefunden hat“, fragen Sie zunächst, ob die Defekte bereits durch den aktuellen Charter abgedeckt waren. Ist dies der Fall, bleibt ihre Korrektur Teil der laufenden Arbeit.

10. Wenn Software funktioniert, aber nichts nützt

Es ist möglich, dass alle technischen Tests grün sind und der CLIENT das Ergebnis dennoch ablehnt. Vielleicht erfordert eine häufige Aktion zu viele Schritte. Vielleicht verwendet die Oberfläche unverständliche Begriffe. Vielleicht ist die Auswertung zwar mathematisch korrekt, beantwortet aber nicht die Frage, die der Vorstand wirklich stellt.

Formulieren Sie eine solche Ablehnung konkret: „Ich kann den Beitragsstand eines Mitglieds finden, aber ich brauche zuerst einen schnellen Überblick über alle überfälligen Beiträge. Die aktuelle Oberfläche hilft bei meiner Hauptaufgabe noch nicht.“ Damit bleibt der fachliche Fortschritt sichtbar und der tatsächliche Mangel wird verständlich.

Anschließend unterscheidet der Planner, ob das ursprünglich vereinbarte Ergebnis nicht erfüllt wurde oder ob eine neue Präferenz hinzugekommen ist. Daraus ergibt sich die richtige Route. Für eine bereits vereinbarte, aber unzureichend umgesetzte Qualität ist meist Nacharbeit innerhalb des bestehenden Auftrags zuständig.

Eine freundliche Bewertung ist keine Pflicht. Die human-perceptual-Prüfung soll tatsächlich feststellen, ob die Software ihren Zweck erfüllt. Das ist besonders wichtig, wenn ein KI-Werkzeug die Umsetzung sehr schnell, aber ohne reale Nutzerinteraktion erzeugt hat.

11. Eine praktische Gesprächsvorlage

Sie können die folgenden fünf Fragen in einer kurzen Besprechung beantworten:

1. Welcher Nutzer erlebt heute welches Problem?
2. Was soll er nach der Änderung konkret besser tun können?
3. Welche bestehenden Daten, Rechte und Abläufe müssen unverändert bleiben?
4. Welcher Termin, welche Kosten oder welches Qualitätsmerkmal sind vorrangig?
5. Welche offene Produktentscheidung würde die Umsetzung grundsätzlich verändern?

Danach soll Engineering den aktuellen Code prüfen, die Folgen erklären, die richtige Fähigkeit abgrenzen und die technischen Risiken behandeln. Nicht jeder Wunsch benötigt ein großes Dokument. Wichtig ist, dass die gültige Entscheidung an der kanonischen Stelle nachvollziehbar wird und sich auf eine reale Abnahme beziehen lässt.

Der beste Änderungsauftrag spart beiden Seiten Arbeit: Der CLIENT beschreibt den gewünschten Nutzen; die technische Rolle verantwortet die Umsetzung und kann begründet zurückfragen, falls das Produkt sonst anders würde als beabsichtigt. Diese Arbeitsteilung ist Kern von ArquitecturaRepo.

Weiter: Kapitel 05, Fehler und Wiederherstellung.

Quellen und Gültigkeit

- CLIENT-Grenze
- Produktentdeckung und technische Baseline
- Planner-Kernel
- Anforderungen und Lieferplan
- Charter-Standard
- Abnahme von Fähigkeiten

Die vorgeschlagenen Wünsche betreffen das fiktive Vereinsbeispiel. Sie sind keine bereits bestätigten Anforderungen für eine fremde Installation.

05 — Fehler, Zwischenfälle und sichere Wiederherstellung

> Fehler sind im Entwicklungsprozess normal. Entscheidend ist, dass bei ihrer Behandlung keine aktuellen Quellen, Daten oder Entscheidungsrechte verloren gehen. Dieses Kapitel ergänzt die Rekonstruktionsregeln, das Recovery-Protokoll und die Sicherheitsgrenzen.

Kapitelübersicht

1. Beweise vor Maßnahmen sichern
2. Umgebungs- und Kontoprobleme
3. Fehler der installierten Steuerungsebene
4. Git ist keine allgemeine Rückgängig-Taste
5. Wiederkehrende Fehler systemisch behandeln
6. Sicherheit, Datenschutz und Rechte Dritter
7. Nachweisbare Rückkehr zum Betrieb
8. Drei vollständige Wiederherstellungsfälle
9. Entscheidungsmatrix für sensible Operationen
10. Was grüne Validierung nicht aussagt
11. Feedback und Telemetrie des Starters

1. Beweise vor Maßnahmen sichern

Die erste Reaktion auf einen Fehler sollte nicht das Löschen oder Neuinstallieren sein. Klären Sie, was tatsächlich fehlgeschlagen ist und welche Informationen für die Diagnose benötigt werden. Notieren Sie die genaue Fehlermeldung, den ausführenden Befehl, den relevanten Pfad, den aktuellen Git-Stand, den beteiligten Datenbankzustand und den Zeitpunkt. Bewahren Sie ungesicherte Änderungen auf.

Je nach Risiko ist die richtige Sofortmaßnahme eine begrenzte Pause. Wenn der Fehler nur ein lokaler fehlender Testimport ist, dürfen unabhängige sichere Arbeiten weiterlaufen. Wenn die Datenbankmigration möglicherweise echte Mitgliedsdaten verändert hat, müssen weitere schreibende Datenbankaktionen angehalten werden, bis die Situation verstanden ist. Der Umfang des HOLD folgt der betroffenen Ressource und nicht einer pauschalen Angst vor jedem Warnsignal.

Vermeiden Sie Aktionen, die die Ursache verschleiern. Ein erzwungener Reset, eine unkontrollierte Neuinstallation, das Löschen aller Caches oder das Ausführen einer Migration ein zweites Mal kann wertvolle Belege zerstören. Bei Datenbanken kann eine Wiederholung sogar denselben wirtschaftlichen Vorgang doppelt verbuchen.

Das Prinzip lautet: Zuerst beobachtbaren Zustand sichern, dann den Fehler eingrenzen, dann eine reversible oder ausdrücklich autorisierte Reparatur durchführen, anschließend den konkreten Effekt nachweisen. Ein plausibler Erklärtext ersetzt dabei weder einen Integritätstest noch die Sichtung des realen Produkts.

2. Umgebungs- und Kontoprobleme

Nicht jeder Fehler liegt im Projektcode. Python kann im falschen virtuellen Environment laufen; eine Abhängigkeit kann fehlen; das Terminal kann sich in einem anderen Arbeitsverzeichnis befinden; eine Anmeldung kann abgelaufen sein. Beginnen Sie bei der tatsächlichen Umgebung und vergleichen Sie erwartete mit beobachteten Werten.

Falls ein Befehl nicht gefunden wird, prüfen Sie den Interpreterpfad und die Version, bevor Sie eine neue Installation starten. Wenn Git kein Repository erkennt, prüfen Sie das Verzeichnis. Wenn ein Agent angeblich keine Dateien sieht, kontrollieren Sie dessen Workspace statt ihm sofort weitere globale Rechte zu geben.

Ein Zugriffsfehler ist keine Einladung, Berechtigungsgrenzen zu umgehen. Wenn ein Dienst ausdrücklich eine neue Anmeldung oder einen gesonderten Zugriff verlangt, muss diese Autorität beim zuständigen Konto eingeholt werden. Keine technischen Tricks und keine ungeprüfte Wiederverwendung fremder Geheimnisse sind angemessene Reparaturen.

Ein externer Anbieter kann außerdem vorübergehend nicht erreichbar sein. Unterscheiden Sie Netzwerkausfall, Tokenproblem, API-Änderung, Quota, Kostenlimit und Programmfehler. Ein Test in der lokalen isolierten Umgebung kann weiter zulässig sein, obwohl produktive externe Aufrufe gesperrt bleiben.

3. Fehler der installierten Steuerungsebene

Die Steuerungsebene .agentic-sdlc/ enthält Verträge und aktuelle Projektbezüge. Schlägt ihre Prüfung fehl, darf man nicht einfach ein fehlendes Feld mit einem erfundenen Wert füllen. Ein Schemafehler, eine widersprüchliche Identität und eine nicht aktuelle Datenbankreferenz sind unterschiedliche Fälle und brauchen unterschiedliche Lösungen.

Ein reiner Formatfehler kann nach Sichtung der beabsichtigten Regel lokal korrigiert werden. Eine unbekannte Autorität erfordert hingegen Rekonstruktion, nicht kosmetische JSON-Bearbeitung. Wenn eine frühere Kampagne behauptet, sie sei abgeschlossen, ihre Quell- oder Review-Referenzen aber fehlen, bleibt die Abnahme unbewiesen. Der Controller kann den erforderlichen Nachweis nachholen oder den Zustand ehrlich als offen kennzeichnen.

Die Werkzeuge project_status.py, validate_project.py und reconstruct_context.py helfen, verschiedene Fehlklassen sichtbar zu machen. Ihre Ausgaben müssen im richtigen Projekt und mit der tatsächlich installierten Version interpretiert werden. Ein grüner Strukturtest beweist nicht, dass die gespeicherten Informationen zur wirklichen Anwendung passen.

Besonders wichtig ist die Regel über aktuelle Quellen: Eine Datei kann formal korrekt und dennoch veraltet sein. Wenn sie auf einen früheren Git-Stand, eine nicht mehr gültige Datenbank oder einen überschriebenen Review verweist, muss diese Referenz geprüft und gegebenenfalls ungültig erklärt werden.

4. Git ist keine allgemeine Rückgängig-Taste

Git kann bekannte Quelländerungen zeigen und frühere Versionen wiederherstellbar machen. Es verwaltet jedoch nicht automatisch alle externen Effekte. Eine Datei kann

zurückgesetzt werden, obwohl eine Datenbankmigration weiterhin wirksam ist. Ein Commit kann wieder entfernt werden, während eine bereits gesendete E-Mail oder eine Zahlung nicht rückgängig gemacht werden kann.

Vor einem Git-Eingriff bestimmen Sie den exakten Zielstand und die betroffenen Änderungen. Wenn die Arbeitskopie ungesicherte Arbeit anderer enthält, dürfen diese Dateien nicht kommentarlos verworfen werden. Ermitteln Sie zuerst ihre Eigentümerschaft. In einem gemeinsam genutzten Repository ist die sichere Wiederherstellung häufig eine gezielte Reparatur oder ein kontrollierter Revert statt eines globalen Resets.

Unterscheiden Sie zudem Git-Autor und reale Ausführungsidentität. Dass ein Commit unter einem bestimmten Namen gespeichert wurde, beweist nicht, welcher Agent die Arbeit tatsächlich ausgeführt oder welche Kampagne sie autorisiert hat. Für sicherheitsrelevante Aktionen braucht es zusätzliche gültige Autoritäts- und Ausführungsbelege.

Temporäre Worktrees können bei echten unabhängigen Arbeiten sinnvoll sein. Bleiben sie jedoch nach Abschluss bestehen, erzeugen sie veraltete Branches und Speicherverbrauch. Entfernen Sie nur jene temporären Arbeitsflächen, deren Zweck und Eigentümerschaft sicher geklärt sind, und erhalten Sie kanonische Artefakte und aktive Zwischenstände.

5. Wiederkehrende Fehler systemisch behandeln

Ein einzelner fehlgeschlagener Test rechtfertigt keine neue strategische Kampagne. I3 untersucht, korrigiert und wiederholt ihn innerhalb des gültigen Charters. Wenn derselbe Fehlertyp aber mehrfach zurückkommt, ist es ein Zeichen dafür, dass die bisherige Diagnose zu eng war.

Nehmen wir eine Mitgliedersuche, bei der nach jeder Korrektur andere Umlaute oder Namensformen scheitern. Statt nacheinander zehn neue Spezialfälle einzubauen, muss I3 die Ursache im ganzen Suchpfad untersuchen: Datenspeicherung, Kodierung, Normalisierung, Vergleichsregeln, Index, API und Darstellung. Eine unabhängige Prüfung sollte dabei bewusst gegen die neue Lösung argumentieren und ungetestete Grenzfälle suchen.

Dies ist eine technische Vertiefung, keine neue Produktdefinition. Erst wenn die Ursache eine bisher unbekannte materielle Produkt- oder Sicherheitsentscheidung erzwingt, wird der Planner erneut eingeschaltet. Reine Implementierungsfehler bleiben in der operativen Verantwortung. Dadurch entstehen weniger unnötige Freigabeschleifen, aber die Qualitätsanforderungen bleiben erhalten.

Auch die Tests selbst können falsch sein. Ein Evaluator, der ein unzulässiges Ergebnis akzeptiert, braucht Reparatur. Ein Evaluator, der eine vollständig erfüllte Fähigkeit wegen einer irrelevant gewordenen Formalie blockiert, ebenso. Die Anpassung muss begründet werden, darf aber nicht heimlich einen fachlich notwendigen Gate entfernen.

6. Sicherheit, Datenschutz und Rechte Dritter

Bei einem Sicherheitsvorfall begrenzen Sie den betroffenen Zugriff und bewahren die relevanten Belege, soweit rechtlich und organisatorisch zulässig. Verschicken Sie keine Geheimnisse in einen öffentlichen Fehlerbericht. Maskieren Sie sensible Werte in Logs und geben Sie nur die für die Diagnose erforderlichen Informationen weiter.

Wenn die Vereinsanwendung echte Kontakt- und Zahlungsdaten enthält, testen Sie Fehler möglichst mit synthetischen oder geeigneten anonymisierten Daten. Die Nutzung realer personenbezogener Daten in einem externen KI-Dienst erfordert eine eigene rechtliche und produktseitige Prüfung. Die bloße Verfügbarkeit einer API ist keine entsprechende Freigabe.

Bei externen Aktionen muss man zwischen reversibler Simulation und realer Wirkung unterscheiden. Ein Test, der eine E-Mail vorbereitet, ist nicht dasselbe wie ihr Versand. Das Erzeugen eines Rechnungsentwurfs ist nicht die Belastung eines Zahlungsmittels. Eine lokale Bildvorschau ist nicht automatisch die öffentliche Veröffentlichung eines fremden Werkes.

Bewerten Sie Lizenzen und Nutzungsrechte angemessen. Ein öffentliches Framework mit MIT-Lizenz kann Drittbibliotheken, Bilder oder Dienste verwenden, deren Bedingungen anders sind. Weder die Nennung eines Links noch ein bestandener Unit-Test ersetzt die Beachtung dieser Bedingungen.

7. Nachweisbare Rückkehr zum Betrieb

Eine Wiederherstellung ist erst abgeschlossen, wenn der relevante Zustand wieder sicher benutzt werden kann. Dies verlangt mehr als das Verschwinden einer Fehlermeldung. Zeigen Sie den korrigierten Kandidaten, die genutzte Sicherung oder Transaktionsquittung, die relevanten bestandenen Prüfungen und die aktuelle Einschränkung.

Nach einer Datenbankkorrektur kontrollieren Sie beispielsweise nicht nur, dass die Anwendung startet. Prüfen Sie, ob alle erwarteten Mitglieder vorhanden sind, Zahlungen nicht doppelt gezählt werden, Berechtigungen weiterhin greifen und ein erneuter Wiederanlauf dieselbe korrekte Wirkung besitzt. Nach einem UI-Fehler kann eine tatsächliche Bedienprobe erforderlich sein.

Der gültige Zustand wird auf der passenden kanonischen Oberfläche festgehalten. Ein alter Fehlerbericht bleibt historische Evidenz, darf aber nicht weiterhin wie der aktuelle Status aussehen. Ebenso darf eine als „recovered“ markierte Kampagne keinen neuen unbestätigten Produktionsstand verbergen.

Wenn die Wiederherstellung nicht sicher abgeschlossen werden kann, dokumentieren Sie präzise, was fehlt. Bleibt beispielsweise unklar, ob eine Zahlung schon extern ausgeführt wurde, dürfen Sie sie nicht erneut auslösen. Die nächste sichere Handlung ist dann ein lesender Nachweis oder eine ausdrücklich koordinierte Prüfung.

8. Drei vollständige Wiederherstellungsfälle

Fall A: Fehlende Python-Abhängigkeit. Nach dem Scaffold schlägt die Validierung wegen eines fehlenden Moduls fehl. Der Entwickler prüft den aktiven Interpreter, die deklarierte Requirements-Datei und den Installationsstatus. Er installiert die dokumentierten Abhängigkeiten im richtigen Environment und wiederholt denselben Prüfpfad. Projektanforderungen, Product State und Kampagnen werden nicht verändert. Das Ergebnis ist eine reparierte lokale Umgebung, keine neue Produktfreigabe.

Fall B: Unterbrochene Datenbankmigration. Eine Testmigration wurde während der Ausführung beendet. I3 stoppt weitere schreibende Zugriffe und prüft Migrationsjournal, Transaktionsstatus, Datenbankschema und verfügbare Sicherung. Anhand dieser Belege entscheidet er, ob die Migration sicher wiederholt,

vervollständigt oder zurückgenommen werden kann. Eine zweite Ausführung ohne Kenntnis der ersten Wirkung ist unzulässig. Erst danach werden Datenintegrität, Startverhalten und relevante Regressionen nachgewiesen.

Fall C: Veralteter Agentenbericht. Ein früherer Bericht nennt die Mitgliedersuche „abgenommen“. Seitdem wurde der Code jedoch geändert, und der neue Stand ist nicht gegen dieselben Voraussetzungen geprüft. Der Planner hält die frühere Abnahme als historischen Beleg fest, erkennt aber fehlende Currentness. I3 prüft den tatsächlich aktuellen Kandidaten erneut und liefert einen frischen, exakt gebundenen Bericht. Der CLIENT prüft die sichtbare Nutzung, wenn das vereinbarte Gate dies verlangt.

Die drei Fälle zeigen unterschiedliche Ebenen: Infrastruktur, persistenter Zustand und Nachweisbindung. Ein einziges pauschales „alles neu starten“ löst diese Probleme nicht zuverlässig. Gute Wiederherstellung reagiert auf den tatsächlichen Wirkungsbereich.

9. Entscheidungsmatrix für sensible Operationen

Geplante Handlung	Übliche technische Route	Zusätzliche Grenze
Logdatei lesen	Lesende Diagnose	Geheimnisse nicht unnötig ausgeben
Unit-Test lokal wiederholen	I3 innerhalb des Charters	Geprüften Kandidaten festhalten
Testdatenbank aus Backup wiederherstellen	I3 nach Zustandsprüfung	Ziel und Wiederholbarkeit eindeutig
Produktionsdatenbank zurücksetzen	Explizite autorisierte Recovery	Datenverlust, Rechte und Sicherungen prüfen
Externe Zahlung erneut senden	Nicht allein aus einem Fehlerbericht	Vorherige Wirkung und CLIENT-Freigabe klären
Fremde Bilder öffentlich nutzen	Rechtliche und produktspezifische Prüfung	Lizenz und Nutzungsumfang beachten
Laufenden Charter umbenennen	Keine gültige Reparatur	Identität und tatsächliche Autorität rekonstruieren
Bereits bestätigte Anforderung ändern	CLIENT und strategischer Eigentümer	Supersession dokumentieren

Diese Matrix ist eine Orientierung. Sie ersetzt keine konkrete Projektpolitik und keine notwendige Rechts- oder Sicherheitsbewertung. Sie hilft aber, alltägliche technische Korrekturen von materiellen externen Eingriffen zu unterscheiden.

10. Was grüne Validierung nicht aussagt

Ein erfolgreicher Test der Steuerungsebene zeigt ihre formale Funktionsfähigkeit innerhalb der geprüften Umgebung. Er zeigt nicht, dass alle Anwendungsfunktionen integriert, alle Daten migrationssicher, alle Rechte korrekt konfiguriert oder alle Oberflächen benutzbar sind. Auch ein erfolgreich erzeugtes PDF oder ein gültiges Manifest beweist nicht, dass ein völlig anderes Produkt richtig arbeitet.

Halten Sie Gates getrennt. Unit prüft kleine technische Einheiten; Integration prüft Zusammenspiel; DB/Currentness prüft aktuelle Zustandsbezüge; semantische Prüfung testet die richtige Bedeutung und Generalisierbarkeit; Physical prüft tatsächliche Ausführung; Perceptual prüft die menschliche Wirkung; Release bewertet die Freigabe des konkreten Pakets. Fehlende Gates dürfen nicht durch ein benachbartes PASS ersetzt werden.

Nicht jede Fähigkeit benötigt jeden denkbaren Gate in maximaler Tiefe. Die Auswahl muss zu Risiko und Nutzung passen. Wer aber ausgerechnet einen relevanten Gate auslässt, muss die entsprechende Unsicherheit offen lassen. So bleibt der Nachweis ehrlich und verhältnismäßig.

11. Feedback und Telemetrie des Starters

Die v1.4-Installation kann standardmäßig einen lokalen Metadatenbericht für Architekturfeedback vorbereiten. Dieser Bericht ist nicht die vollständige Quellcodeübertragung an einen Maintainer. Der periodische Workflow ist von der lokalen Berichtserstellung getrennt, und eine externe Sammlung wird dadurch nicht automatisch eingerichtet.

Falls keinerlei Berichtserstellung gewünscht wird, setzen Sie in der installierten Datei .agentic-sdlc/feedback/ARCHITECTURE_FEEDBACK_CONFIG.json den Wert enabled auf false. Der Parameter –no-feedback-workflow deaktiviert lediglich die Zeitplanung. Lesen Sie die Datenschutzregel, bevor Sie die Einstellung ändern.

Bei einem vermeintlichen Telemetrieproblem prüfen Sie zunächst den tatsächlichen Ausführungspfad und den Inhalt des erzeugten lokalen Berichts. Eine geplante GitHub-Actions-Ausführung im eigenen Repository ist nicht dasselbe wie eine Übermittlung an einen fremden Server. Behaupten Sie weder eine externe Datenübertragung noch deren Abwesenheit, ohne den realen Pfad zu untersuchen.

Eine sichere Incident-Kultur schützt Daten und Belege, ohne jeden Alltagsschritt durch formale Eskalationen zu blockieren. Sie erkennt den ursächlichen Eigentümer, repariert dessen System und lässt eine neue Sitzung die echte Fortsetzung rekonstruieren.

Weiter: Kapitel 06, Fähigkeiten abschließen.

Quellen und Gültigkeit

- Currentness und Rekonstruktion
- Transaktionswiederherstellung
- Sicherheit und externe Aktionen
- Repository- und Worktree-Hygiene
- Review- und Terminalintegrität
- Architekturfeedback und Datenschutz
- Zustandsprüfung

Die Beispiele verwenden fiktive Vereinsdaten und begründen keine Schreibberechtigung für produktive Systeme.

06 — Fähigkeiten abnehmen und nachvollziehbar abschließen

> Eine Fähigkeit ist nicht fertig, weil ein Ausführender „PASS“ schreibt. Dieses Kapitel zeigt, wie eine konkrete Produktwirkung an einen aktuellen Kandidaten, getrennte Nachweise und die richtige Entscheidungsautorität gebunden wird. Maßgeblich sind der Capability-Lebenszyklus und die Integrität des Reviews.

Kapitelübersicht

1. Eine Fähigkeit ist ein benutzbares Ergebnis
2. Fünf Bausteine eines Abschlusses
3. Der genaue Kandidat als Prüfgegenstand
4. Evidenzlücken ohne künstliche neue Kampagne behandeln
5. Annehmen, fortsetzen oder das Ziel ändern
6. Regelmäßige Prüfung und vernünftige Stoppregel
7. Ein überprüfbarer Abschlussdatensatz
8. Zurückweisung und Korrektur im bestehenden Auftrag
9. Brauchbare Software und grünes Framework
10. Wann Arbeit wirklich beendet werden kann

1. Eine Fähigkeit ist ein benutzbares Ergebnis

Ein Feature wird häufig in technischen Artefakten beschrieben: ein neuer Endpunkt, eine Datenbanktabelle oder eine Komponente. Für die Produktabnahme genügt diese Sicht nicht. Eine Fähigkeit ist das beobachtbare Ergebnis, das der CLIENT erwartet. Bei einer Vereinsanwendung ist dies beispielsweise die Möglichkeit, einen Mitgliedsbeitrag korrekt zu erfassen, wiederzufinden und im späteren Verlauf unverändert nachzuweisen.

Die Datenbanktabelle kann existieren, obwohl das Formular falsche Beträge zulässt. Der Endpunkt kann alle Unit-Tests bestehen, obwohl die Oberfläche den falschen Mitgliedsdatensatz auswählt. Ein Screenshot kann gut aussehen, obwohl die Speicherung bei einem Neustart verloren geht. Ein erfolgreicher Abschluss muss deshalb den gesamten relevanten Arbeitsweg berücksichtigen.

Das bedeutet nicht, jede denkbare Zukunftsanforderung zu testen. Die Fähigkeit wird gegen ihren ausdrücklich vereinbarten Umfang bewertet. Was ausgeschlossen oder erst für eine spätere Version vorgesehen ist, darf nicht stillschweigend zum aktuellen Abnahmeproblem werden. Gleichzeitig darf ein notwendiger Sicherheits- oder Integritätsnachweis nicht durch die Behauptung „nicht im Scope“ unterdrückt werden, wenn die aktuelle Fähigkeit ohne ihn gefährlich wäre.

Ein guter Abschluss beantwortet: Was kann der Nutzer jetzt tatsächlich tun? Mit welchem konkreten Code- und Datenstand? Unter welchen Bedingungen? Was wurde unabhängig geprüft? Welche Einschränkungen bleiben? Wer hat welche Form der Annahme ausgesprochen?

2. Fünf Bausteine eines Abschlusses

Für eine klare Prüfung helfen fünf eng verbundene Bausteine. Erstens muss die vereinbarte Produktfähigkeit wiedergefunden werden können: Anforderung, Charter und zulässige Grenzen. Zweitens muss der genaue technische Kandidat identifizierbar sein. Drittens braucht der Kandidat passende technische, semantische und physische Nachweise. Viertens müssen unabhängige Review- und gegebenenfalls menschliche Qualitätsentscheidungen vorliegen. Fünftens ist der aktuelle Zustand mitsamt den offenen Einschränkungen dauerhaft zu dokumentieren.

Diese Bausteine sind keine fünf zusätzlichen Bürokratieschritte. Häufig entstehen sie aus ohnehin notwendigen Arbeiten. Der Entwickler kennt den Code, die Tests laufen auf einem konkreten Stand, ein Reviewer dokumentiert seine Befunde und der CLIENT sieht eine Demonstration. Das Framework hilft, diese Informationen zusammenzuführen, sodass sie später noch überprüfbar sind.

Ein valider Beleg kann ein Testprotokoll, eine Datenbankquittung, eine lauffähige Anwendung, eine Bildschirmaufnahme oder eine unabhängige Review-Notiz sein. Welcher Beleg angemessen ist, hängt von der Behauptung ab. Ein Git-Commit allein kann eine Datenbankmigration nicht nachweisen; ein Datenbanksnapshot beweist keine gute Bedienbarkeit; ein hübsches Video keine sichere Berechtigungsprüfung.

Legen Sie dabei Wert auf Kausalität. Der Test muss tatsächlich gegen den Kandidaten ausgeführt worden sein, der zur Annahme steht. Eine ältere Testliste, die nach einem wichtigen Codeumbau erneut angehängt wird, ist keine frische Prüfung. Wenn eine materielle Änderung nach dem Review erfolgt, müssen die betroffenen Nachweise erneuert werden.

3. Der genaue Kandidat als Prüfgegenstand

Bei veränderlichen Projekten ist die Identität des Kandidaten zentral. Sie kann einen exakten Git-Stand, relevante Konfiguration, Datenbankschema, Migrationen, Laufzeitumgebung und die tatsächlichen Artefakte umfassen. Es reicht nicht, „neueste Version“ zu schreiben, wenn sich der Branch ständig weiterbewegt.

Der Review-Freeze hält den konkreten Gegenstand für die Prüfung fest. Eine unabhängige Person soll denselben Stand untersuchen, nicht einen inzwischen veränderten Arbeitsbaum. Werden danach wichtige Codeänderungen vorgenommen, wird ein neuer Kandidat erstellt und der relevante Review wiederholt. Kleinere Dokumentationskorrekturen können ein anderes Risiko haben; maßgeblich ist ihre tatsächliche Auswirkung auf die geprüfte Fähigkeit.

Ein Beispiel: Der Controller liefert die Mitgliedersuche aus Commit A und ein Reviewer prüft sie auf einer bekannten Testdatenbank. Nach dem Review repariert der Controller eine Zugriffsprüfung in Commit B. Der vorherige Review von A kann nützlich bleiben, belegt aber nicht automatisch B. Die neue Sicherheitslogik verlangt passende frische Tests und eine erneute unabhängige Betrachtung der betroffenen Stelle.

Git allein ist außerdem kein Beweis, wer als verantwortlicher Akteur handelte. Halten Sie Freigaben und Ausführungsidentität auf den dafür vorgesehenen Oberflächen fest, wenn dies für den Kandidaten relevant ist. Die öffentliche Architektur unterscheidet bewusst Commitgeschichte, technische Arbeit und Entscheidungsautorität.

4. Evidenzlücken ohne künstliche neue Kampagne behandeln

Wenn eine Abnahmeprüfung eine fehlende Testart oder einen Implementierungsfehler entdeckt, wird der Befund innerhalb des geltenden Charters an I3 zurückgegeben. Der Controller repariert die Ursache, ergänzt den notwendigen Nachweis und liefert einen aktualisierten Kandidaten. Es ist keine neue Planungskampagne nötig, solange die ursprüngliche Produktabsicht und der autorisierte Umfang weiterhin gelten.

Angenommen, alle Unit-Tests der Beitragsfunktion bestehen, aber der tatsächliche Neustart der Anwendung wurde noch nicht geprüft. Das fehlende Physical-Gate kann in derselben Kampagne nachgeholt werden. Falls dabei ein Persistenzfehler auftritt, korrigiert I3 ihn. Ein neuer PASS1/PASS2-Zyklus allein wegen dieser gewöhnlichen Fehler würde nur unnötige Planungsarbeit erzeugen.

Anders ist die Situation, wenn die reale Nutzung zeigt, dass die Fähigkeit einen völlig anderen Sicherheitsrahmen benötigt. Eine lokale Vereinsdatenbank soll plötzlich direkt aus dem öffentlichen Internet erreichbar werden. Das verändert Architektur, Bedrohungsmodell, Betrieb und möglicherweise Datenverarbeitung. Der Planner muss diese neue strategische Lage überprüfen, statt sie als kleinen In-Scope-Bug zu behandeln.

Ein zweiter Sonderfall ist ein dauerhaft falscher Evaluator. Wenn er Belege akzeptiert, die die versprochene Wirkung gar nicht zeigen, muss der Bewertungsmechanismus korrigiert werden. Wenn er einen nachweislich erfüllten und angemessenen Auftrag aus rein formalen Gründen dauerhaft blockiert, muss ebenfalls seine Ursache geprüft werden. Die Reparatur darf keinen notwendigen Gate entfernen, sondern soll ihn wieder sinnvoll machen.

5. Annehmen, fortsetzen oder das Ziel ändern

Nach einem konsolidierten Bericht gibt es drei wesentliche Routen. **Annehmen**, wenn der aktuelle Kandidat die vereinbarten Bedingungen mit geeigneter Evidenz erfüllt und alle reservierten Entscheidungen vorliegen. **Fortsetzen**, wenn gewöhnliche Fehler oder Beweislücken innerhalb des gültigen Charters bestehen. **Strategisch ändern**, wenn neue Produktabsicht oder materielle Risikolage die ursprüngliche Vereinbarung ungültig gemacht haben.

Diese Routen sollten nicht vermischt werden. Ein defektes Formular verlangt kein neues Produktziel. Ein komplett neuer öffentlicher Zahlungskanal darf dagegen nicht einfach innerhalb des alten lokalen Beitrags-Chapters gebaut werden. Die richtige Zuordnung schützt sowohl Fortschritt als auch Auftraggeberkontrolle.

Eine Ablehnung sollte konkrete Falsifizierer nennen. Zum Beispiel: „Bei zwei gleichzeitigen Speichervorgängen entsteht ein doppelter Beitrag; die Datenbankprüfung des exakten Kandidaten zeigt das.“ Das ist für I3 ausführbar. „Ich habe ein schlechtes Gefühl, macht alles noch einmal“ ist als technische Anweisung unzureichend, kann aber ein Anlass sein, eine menschliche UX-Demonstration zu verlangen, wenn der Eindruck tatsächlich die Produktqualität betrifft.

Wenn eine Fähigkeit bewusst mit Einschränkungen angenommen wird, müssen diese Einschränkungen eindeutig dokumentiert werden. Eine bedingte Freigabe ist keine heimliche Behauptung, alles sei gelöst. Ein offenes Risiko mit realer Datenverlustmöglichkeit kann zudem eine Annahme ausschließen, selbst wenn die Oberfläche gut wirkt.

6. Regelmäßige Prüfung und vernünftige Stoppregel

Eine Qualitätsprüfung muss streng genug für das echte Risiko sein. Sie darf jedoch nicht zu einer endlosen Suche nach theoretischen Mikroproblemen werden, wenn alle strategisch relevanten Grenzen bereits geklärt und die vereinbarten Gates erfüllt sind. Ziel ist nutzbare, verantwortbar geprüfte Software, nicht die größtmögliche Zahl von Reviewrunden.

Der Planner kann regelmäßige Zwischenkontrollen nutzen, um Richtung, Abhängigkeiten und Risiken früh zu erkennen. Solche Kontrollen sind keine vollständigen erneuten strategischen PASS-Zyklen. Ein formaler Review am Kampagnenende bewertet den gesamten Kandidaten und seine Nachweise, nicht bloß das zuletzt bearbeitete kleine Ticket.

Wenn zwei oder mehr Rückläufe dieselbe Fehlklasse betreffen, sollte I3 nicht weiter einzelne Symptome „abbauen“. Er führt eine umfassende technische Härtung und eine unabhängige Überprüfung des betroffenen Subsystems durch. Erst danach ist ein weiterer konsolidierter terminaler Vorschlag sinnvoll. Diese Stoppregel richtet sich gegen ineffizientes Churn, nicht gegen notwendige Tests.

Fortschritt messen Sie am Freischalten einer Fähigkeit, am erfolgreichen Durchlaufen eines relevanten Gates und an real verringerter Gefahr. Die Anzahl von Commits, Agenten und Dokumenten ist nicht entscheidend. Ein einziges behobenes Datenintegritätsproblen kann wertvoller sein als hundert neue Formatchecks.

7. Ein überprüfbarer Abschlussdatensatz

Ein kompakter Abschlussbericht für die Beitragsfunktion könnte folgende Inhalte haben:

Feld	Beispiel einer überprüfbaren Aussage
Fähigkeit	Beitragszahlung erfassen und historisch wiederfinden
Produktgrenze	Nur lokale Vereinsverwaltung, kein externer Zahlungseinzug
Kandidat	Exakter Repository-Stand samt Datenbankschema und relevanten Artefakten
Unit	Betragsvalidierung, Rundung und Pflichtfelder geprüft
Integration	Formular, Service und Datenbank gemeinsam geprüft
DB/Currentness	Migration und gespeicherte Zahlungen entsprechen dem aktuellen Kandidaten
Semantic	Doppelte Buchung und Mitgliedsdeaktivierung fachlich richtig behandelt
Physical	Anwendung unter den vorgesehenen Betriebsbedingungen gestartet
Perceptual	Verwaltungskraft konnte den vereinbarten Ablauf verstehen und ausführen

Feld	Beispiel einer überprüfbaren Aussage
Release	Konkretes Paket und Veröffentlichungsberechtigung separat beurteilt
Offenes Risiko	Nur wenn tatsächlich vorhanden, mit Auswirkung und Eigentümer

Die Tabelle sind Beispiele für notwendige Informationen, keine Behauptung, dass der fiktive Verein diese Tests schon durchgeführt hat. In einem realen Bericht müssen die Werte auf tatsächliche Ausgaben und überprüfbare Kennungen zeigen. Eine Rubrik „bestanden“ ohne Quelle ist ein unbelegter Status.

Der Abschluss muss ferner die nächste zulässige Handlung enthalten. Nach der Annahme einer Funktion kann die nächste Fähigkeit eine Beitragsübersicht sein. Falls ein Gate offen bleibt, muss sichtbar sein, wer die Nacharbeit erledigt und welche produktiven Aktionen vorerst nicht erlaubt sind.

8. Zurückweisung und Korrektur im bestehenden Auftrag

Ein Reviewer entdeckt, dass die Beitragsfunktion bei einer Datenbankunterbrechung gelegentlich denselben Zahlungseintrag doppelt schreibt. Der geltende Charter verlangt korrekte, einmalige Verbuchung. Der Befund ist damit eindeutig in-scope.

I3 untersucht den vollständigen Buchungspfad: UI-Ereignisse, Retry-Verhalten, Service-Logik, Transaktionen, eindeutige Schlüssel und Fehlerbehandlung. Nach der Korrektur wiederholt er Tests mit Unterbrechungen und identischen Anfragen. Eine unabhängige Prüfung betrachtet anschließend die neu eingeführte Idempotenzmechanik.

Der Planner erhält einen konsolidierten Bericht für den nun aktuellen Kandidaten. Er braucht nicht für jede technische Untersuchung eine neue Kampagne zu autorisieren. Der CLIENT muss auch keine Datenbankstrategie aussuchen. Seine relevante Entscheidung betrifft weiterhin die fachliche Bedeutung einer Zahlung und die Zulässigkeit externer Effekte.

Falls die Untersuchung unerwartet zeigt, dass der Verein jetzt echte Banktransaktionen über eine externe API erwartet, ist das keine reine Reparatur. Diese neue Produktabsicht wird gesondert bestätigt und strategisch bewertet. Nur dann entsteht eine neue Scope-Entscheidung.

9. Brauchbare Software und grünes Framework

ArquitecturaRepo kann eigene Strukturtests erfolgreich bestehen, während eine fremde Anwendung fehlerhaft bleibt. Die Werkzeuge prüfen nur, wofür sie gebaut wurden. Ein funktionierender Kampagnencontroller erzeugt noch keine nutzbare Vereinsverwaltung. Ein fertig gestaltetes Handbuch und ein gültiger Release-Hash verbessern die Dokumentation des Starters, aber nicht automatisch die Produkte seiner Nutzer.

Umgekehrt kann eine Anwendung in einer konkreten Demonstration gut aussehen, obwohl ihre Autoritäts- oder Datenbelege unzureichend sind. Beides muss im passenden Verhältnis geprüft werden. Der CLIENT darf die Nutzerqualität beurteilen, während Engineering den technischen Sicherheits- und Integritätsnachweis verantwortet.

Wichtig ist die Fähigkeit, eine ehrliche Grenze zu setzen. „Unter Testdaten erfolgreich, noch nicht für Produktion freigegeben“ ist eine korrekte Aussage. „Der Test ist grün, daher können wir veröffentlichen“ kann falsch sein. Gute Abnahme erhöht Vertrauen gerade dadurch, dass sie Aussagen nur so weit macht wie die Evidenz trägt.

10. Wann Arbeit wirklich beendet werden kann

Eine Kampagne kann geschlossen werden, wenn ihre vereinbarte Fähigkeit mit den zutreffenden technischen und menschlichen Nachweisen akzeptiert ist und der aktuelle Zustand sauber hinterlassen wurde. Eine Arbeitssitzung kann auch beendet werden, obwohl die Kampagne noch offen ist, sofern ein sicherer Checkpoint mit konkreter Fortsetzung existiert.

Der CLIENT muss nicht ständig online sein. Fehlt eine neue, wirklich notwendige Produktfreigabe, wird die davon abhängige Handlung angehalten. Andere technisch unabhängige Arbeiten können im zulässigen Umfang fortlaufen. Ein bloßes Ausbleiben der Antwort ist keine neue Produktentscheidung und kein stillschweigender Auftrag, laufende Arbeit zu stoppen.

Abschließend sollte eine frische Sitzung die angenommene Fähigkeit, den Kandidaten, die Prüfungen, offene Risiken und die nächste Route rekonstruieren können. Diese Fähigkeit zur sicheren Fortsetzung ist selbst ein Qualitätsmerkmal des gesamten Entwicklungsprozesses.

Weiter: Kapitel 07, durchgearbeitetes Vereinsbeispiel.

Quellen und Gültigkeit

- Lebenszyklus und Nachweise einer Fähigkeit
- Review-Freeze und Terminalintegrität
- Kampagnen-Charter
- Execution Team OS
- Release-Kriterien
- Rekonstruktionsregeln

Die tabellarischen Belege sind Demonstrationsmuster; echte Abnahme verlangt nachgewiesene Ergebnisse des eigenen Projekts.

07 — Durchgearbeitetes Beispiel: Vereinsverwalt

> Dieses letzte Kapitel führt eine fiktive Anwendung vom ersten Wunsch bis zur nachvollziehbaren Annahme. Alle Namen, Daten und Ergebnisse sind didaktische Beispiele, keine behauptete Ausführung. Die Referenzanwendung belegt ausgewählte Mechanismen des Starters und ist nicht mit einer echten Produktfreigabe gleichzusetzen.

Der Weg durch das Beispiel

1. Das Produkt und seine tatsächlichen Nutzer
2. Fragen, die die Lösung verändern
3. Den Starter sicher installieren
4. Erste Fähigkeit: Mitglieder verwalten
5. Vom Planner zur ausführbaren Kampagne
6. Fachliche und technische Tests
7. Zweite Fähigkeit: Vereinsbeiträge
8. Das Gesamtprodukt bewerten
9. Erster Entwicklungszyklus mit fiktiven Daten
10. Einen Befund an die richtige Rolle routen
11. Zweiter Zyklus mit klaren Abhängigkeiten
12. Übung: Fortsetzung in einem neuen Kontext

1. Das Produkt und seine tatsächlichen Nutzer

Stellen Sie sich einen örtlichen Sportverein mit einer Verwaltungskraft und einigen hundert Mitgliedern vor. Bislang führt die Verwaltung ihre Mitglieder und Beitragszahlungen in Tabellen. Unterschiedliche Dateiversionen verursachen gelegentlich doppelte Einträge. Der Vorstand möchte wissen, welche Beiträge bezahlt wurden; die Verwaltung benötigt vor allem eine einfache Erfassung und eine schnelle Suche.

Der Auftraggeber formuliert den Zweck: „Wir möchten Mitglieder zuverlässig verwalten und sehen, welche Beiträge offen sind. Die erste Version läuft auf einem vorhandenen Vereinsrechner. Sie soll leicht verständlich sein. Externe Zahlungsanbieter brauchen wir noch nicht.“

Damit sind erste Grenzen gesetzt. Der Nutzen besteht nicht darin, möglichst viele APIs oder KI-Agenten einzusetzen. Er besteht in einem korrekten, benutzbaren Arbeitsablauf für die Verwaltung und in einer nachvollziehbaren Auswertung für den Vorstand. Das konkrete Datenmodell, die Programmiersprache und die Testorganisation bleiben technische Entscheidungen.

Der Auftraggeber darf außerdem entscheiden, welche Daten gespeichert werden. Namen und Kontaktinformationen sind potenziell personenbezogen. Es muss geklärt werden, wer sie lesen darf, ob sie länger aufbewahrt werden und wie die Anwendung gesichert wird. Die fiktiven Beispieldaten, die wir hier verwenden, enthalten keine realen Vereinsmitglieder.

Ein Liefertermin könnte vereinbart werden, doch er ersetzt keine Definition des ersten brauchbaren Ergebnisses. Für den Verein ist eine funktionierende Mitgliedererfassung mit Suche nützlicher als ein technisch beeindruckendes Dashboard ohne verlässliche Datengrundlage.

2. Fragen, die die Lösung verändern

Der Planner führt ein kurzes Produktgespräch. Er fragt nicht nach dem bevorzugten Datenbankindex. Er will wissen, was fachlich geschieht:

- Welche Daten sind beim Anlegen eines Mitglieds zwingend notwendig?
- Können zwei Mitglieder denselben Namen haben?
- Was bedeutet „deaktiviert“ für historische Beiträge und Aktivitäten?
- Wer darf Mitgliedsdaten ändern und wer darf nur Berichte sehen?
- Soll die Anwendung ohne Netzwerkzugang funktionieren?
- Was passiert, wenn eine Beitragszahlung versehentlich zweimal eingegeben wird?
- Wie lange darf die Wiederherstellung nach einem Rechnerausfall dauern?
- Welche Funktionen sind für die erste Vorführung unverzichtbar?

Angenommen, der CLIENT bestätigt: Eine Verwaltungskraft arbeitet lokal, Mitglieder benötigen eine stabile interne Identität, Zahlungen bleiben nach Deaktivierung nachvollziehbar, und die erste Version hat keine Onlinezahlung. Die Anwendung muss verständliche Fehlermeldungen zeigen und vor doppelten Mitgliedern warnen. Über ein konkretes Backup-Intervall entscheidet der CLIENT noch nicht; der Punkt bleibt offen.

Diese Antworten werden als aktuelle Anforderungen dokumentiert. Die technische Rolle darf zusätzliche Folgen erklären: Eine stabile ID verhindert, dass Namensänderungen die Zahlungsverknüpfung zerstören; eine Historie verlangt ein geeignetes Persistenzmodell; fehlender Netzwerkzugang begünstigt lokale Speicherung. Die Entscheidung über Tabellen, Frameworks und Sicherungsmechanismen trifft Engineering.

Der Planner achtet darauf, die Beispielantworten nicht als reale Kundenfreigaben zu behandeln. In einer echten Installation müssen sie vom tatsächlichen CLIENT kommen und auf den aktuell gültigen Zustand angewendet werden.

3. Den Starter sicher installieren

Das Team besitzt bereits ein separates Anwendungsrepository verein-app. Der öffentliche Starter ArquitecturaRepo wurde als v1.4.0-Release bezogen und gegen die veröffentlichte Prüfsumme kontrolliert. Beide Verzeichnisse bleiben getrennt. Die Installation wird nicht im Starter selbst ausgeführt, um dort Vereinsfunktionen zu entwickeln.

Für ein neues, noch nicht geregeltes Projekt mit dauerhaften Mitglieds- und Zahlungsdaten entscheidet das Team sich begründet für STATEFUL. Im geprüften Starter könnte der Befehl lauten:

```
& "python" "tools\scaffold_project.py" --target "..\verein-app" --profile "STATEFUL" --product-id "VEREIN_APP" --objective "Lokale Mitglieder- und Beitragsverwaltung" --language "de" --no-feedback-workflow
```

Danach liegen die generierten Kontrollunterlagen in verein-app/.agentic-sdlc/. Die

anfängliche Anzeige HOLD / UNTRUSTED_CONTEXT ist richtig. Der Starter kennt noch keinen echten Datenbankstand, keinen angenommenen Charter und keine bestätigte Produktbeschreibung. Ein Assistent darf diese Informationen nicht aus dem Beispiel übernehmen.

Im Zielprojekt werden die vorgesehenen Anforderungen installiert und der lesende Prüfpfad ausgeführt:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-
sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-
sdlc"
```

Stellen Sie sich vor, dass die Strukturprüfung gelingt, die Rekonstruktion aber noch gültige Anforderungen vermisst. Das ist kein Widerspruch: Die Ordner sind vorhanden, doch die echte Projektübernahme steht noch aus. Der Planner führt nun Discovery durch, verifiziert den Code und die Betriebsumgebung und holt die notwendige Produktbestätigung ein.

Wäre verein-app bereits von einem anderen System mit gültigen Steuerungsregeln verwaltet, wäre das obige Scaffold nicht die richtige Route. Dann müsste die bestehende Governance lesend geprüft werden, bevor man etwas verändert.

4. Erste Fähigkeit: Mitglieder verwalten

Der Planner wählt eine überschaubare, aber nutzbare erste Fähigkeit: „Mitglied anlegen, später anhand einer stabilen Kennung wiederfinden, wichtige Daten ändern und eine Person deaktivieren, ohne ihre historische Identität zu verlieren.“ Dieser Umfang umfasst mehr als das bloße Erstellen eines Formulars.

Die fachlichen Bedingungen könnten so aussehen:

Vorgang	Beobachtbarer Erfolg	Wichtiger Grenzfall
Anlegen	Neues Mitglied erhält eine stabile ID und ist wiederfindbar	Fehlende Pflichtdaten
Suchen	Berechtigte Verwaltung findet nach Name oder ID	Gleicher Name bei verschiedenen Personen
Bearbeiten	Änderungen sind nach erneutem Öffnen vorhanden	Gleichzeitige Änderung, soweit im Scope
Deaktivieren	Status wechselt ohne Löschung der Zahlungsbezüge	Versuch, historische Belege zu entfernen
Fehleranzeige	Nutzer versteht, was er korrigieren muss	Doppelte Eingabe oder Speicherausfall

Noch ist kein Test bestanden. Die Tabelle beschreibt, was nachgewiesen werden soll. Engineering bestimmt nun die technische Basis und erstellt einen kurzen Lieferplan. Beispielsweise werden zuerst Stammdaten und Persistenz, danach die Suchoberfläche

und zum Schluss Deaktivierung samt Regressionen integriert. Diese Reihenfolge kann sich nach echter Codeinspektion ändern.

Der CLIENT bestätigt die fachliche Bedeutung und den sichtbaren Qualitätsanspruch. Er muss nicht entscheiden, wie Datenbanktransaktionen implementiert werden. Wenn die technische Rolle ein Risiko identifiziert, etwa den möglichen Verlust von Mitgliederdaten bei Stromausfall, erklärt sie die Produktfolge und verlangt nur die dafür notwendige fachliche Entscheidung.

5. Vom Planner zur ausführbaren Kampagne

Der Planner führt seine strategische Hauptbetrachtung durch: Ziel, Nutzerfluss, Daten, Schnittstellen, Sicherheitsrisiken, Fehlerfälle, Wiederherstellung, Abhängigkeiten und notwendige Gates. Eine separate adversariale Betrachtung sucht nach falschen Annahmen. Erst danach entsteht der Capability-Charter, der die erste Mitgliederfähigkeit beschreibt.

Der Charter enthält das erwartete Ergebnis, die Ausgangsquellen, den zulässigen Umfang, seine Schnittstellen, die Verantwortung von I3 und die Abnahmebedingungen. Er muss nicht jede interne Funktion und jeden Git-Commit vorgeben. Mit seiner Freigabe übernimmt der Controller die operative Umsetzung.

Eine kompakte Übergabe könnte so formuliert sein:

> Ziel: Die bestätigte lokale Mitgliederverwaltung end-to-end nutzbar machen. Ausgangsquellen: aktueller geprüfter Anwendungsstand, Product State und bestätigte Anforderungen. Grenzen: keine externen Provider, keine realen Daten versenden, kein öffentliches Deployment, keine Onlinezahlung. Nachweise: Unit, Integration, DB/Currentness, fachliche Grenzfälle, reale lokale Bedienprobe und vereinbarte menschliche Qualität. Abschluss: exakten Kandidaten, Fehler, Tests und offene Risiken vorlegen.

Dies ist keine Einladung zur unbegrenzten autonomen Ausführung. Die Ausführung muss in der tatsächlich autorisierten Umgebung mit den passenden Rechten erfolgen. Ein optionaler lokaler Controller kann Zustände verwalten, aber seine Existenz bedeutet nicht, dass ein externer Agent bereits gestartet wurde.

6. Fachliche und technische Tests

Die Implementierung kann aus einer Datenhaltung, einem Service und einer Oberfläche bestehen. Es ist nicht notwendig, eine bestimmte Architektur zu erzwingen, solange sie die bestätigten Anforderungen zuverlässig erfüllt und angemessene technische Qualität bietet. Die ersten Unit-Tests prüfen etwa Pflichtfelder, Identitätsregeln und Statuswechsel.

Integrationstests prüfen anschließend den vollständigen Weg vom Formulareintrag bis zur gespeicherten und erneut gelesenen Person. Ein separat geprüfter Datenbankstand muss zu den tatsächlich verwendeten Migrationen passen. Semantic-Tests verwenden unterschiedliche Namen, leere Felder, Sonderzeichen und doppelte Personen, um fachliche Generalisierbarkeit zu untersuchen.

Physical bedeutet: Die Anwendung wird unter den vorgesehenen Bedingungen tatsächlich gestartet und bedient. Wenn ein automatisierter Test nur einzelne Funktionen importiert, ersetzt er diesen Schritt nicht. Perceptual bedeutet: Ein vorgesehener Nutzer versteht, welche Eingabe erwartet wird, was ein Fehler

bedeutet und wie er einen Datensatz wiederfindet. Diese Prüfung kann nicht durch einen bloßen Quellcodekommentar ersetzt werden.

Angenommen, die Speichertests bestehen, aber die Oberfläche zeigt nach dem Anlegen keine Bestätigung. Das kann ein echtes Benutzbarkeitsproblem sein. I3 korrigiert es im laufenden Charter, wenn eine verständliche Rückmeldung bereits Bestandteil der Abnahme war. Eine zusätzliche neue Kampagne würde nichts verbessern.

Die Tests und die menschliche Prüfung müssen den exakten Kandidaten betreffen. Wird danach wichtiges Verhalten verändert, sind die betroffenen Nachweise zu erneuern. Ein Bericht über einen älteren Stand ist nicht automatisch ein gültiges Review für die aktuelle Version.

7. Zweite Fähigkeit: Vereinsbeiträge

Erst wenn die Mitgliederidentität verlässlich ist, wird die nächste Fähigkeit sinnvoll: Beiträge erfassen und den Verlauf anzeigen. Diese Fähigkeit hängt von bestehenden Mitgliedern, ihrer stabilen ID und den Regeln für deaktivierte Personen ab. Die Reihenfolge schützt vor einer losgelösten Zahlungstabelle ohne zuverlässige Verknüpfung.

Der CLIENT entscheidet, was als Beitrag gilt, ob Teilzahlungen möglich sind, welche Beträge erwartet werden und wer Korrekturen vornehmen darf. Das Team erklärt die Folgen für Datenmodell und Auswertungen. Externe Bezahlvorgänge bleiben ausdrücklich außerhalb des ersten Umfangs. Ein lokal erfasster Beitrag ist kein Beweis für eine echte Banktransaktion.

Ein wichtiges Risiko ist doppelte Verbuchung. Wenn die Verwaltungskraft auf „Speichern“ doppelt klickt oder eine Netzwerkstörung später einmal zu einem Retry führt, darf aus einer fachlich einmaligen Zahlung nicht unbemerkt eine doppelte Buchung werden. Engineering gestaltet dafür geeignete Identitäten, Transaktionen und Testfälle.

Auch die Historie muss erhalten bleiben. Wenn ein Mitglied deaktiviert wird, sollen seine bisherigen Beiträge nicht verschwinden. Eine Namensänderung darf historische Buchungen nicht einer anderen Person zuordnen. Der zweite Charter baut also auf nachgewiesenen Eigenschaften des ersten auf; er erfindet deren Abnahme nicht neu.

8. Das Gesamtprodukt bewerten

Nach zwei Fähigkeiten soll die Verwaltung einen vollständigen Ablauf demonstrieren können: ein Mitglied anlegen, die Person suchen, einen Beitrag erfassen, eine vergangene Zahlung wiederfinden und eine Person deaktivieren, ohne ihre Historie zu verlieren. Diese zusammenhängende Probe ist wertvoller als zwei isolierte grüne Modulberichte.

Der Planner fragt nach den übergreifenden Risiken: Stimmen Datenmodell und Oberfläche überein? Verhindern Berechtigungen unzulässige Änderungen? Sind Daten nach einem Neustart noch vorhanden? Ist die Wiederherstellung getestet, soweit sie vereinbart wurde? Kann der CLIENT seine häufigsten Aufgaben tatsächlich ausführen?

Eine gute Bewertung erkennt ebenfalls Grenzen. Ein lokaler, kleiner Vereinsbetrieb ist keine automatische Freigabe für tausende gleichzeitige Internetnutzer. Ein Test

mit synthetischen Zahlungsdaten ist keine Abnahme eines externen Finanzdienstes. Die erreichte Fähigkeit darf genau beschrieben werden, ohne ihre Gültigkeit zu überdehnen.

Der Erfolg besteht in einer praktisch verwendbaren Grundlage und einem vertrauenswürdigen nächsten Schritt. Mehr Dokumente oder zusätzliche Agenten sind nur dann sinnvoll, wenn sie diese Ziele unterstützen.

9. Erster Entwicklungszyklus mit fiktiven Daten

Für einen realitätsnahen Probezyklus legen Sie synthetische Mitglieder an: Person A mit aktivem Status, Person B mit gleichem Nachnamen, Person C mit späterer Deaktivierung. Führen Sie erfolgreiche und ungültige Eingaben aus. Testen Sie bewusst Sonderzeichen im Namen, fehlende Pflichtwerte und eine Korrektur der Kontaktadresse.

Danach schließen Sie die Anwendung vollständig und öffnen sie erneut. Prüfen Sie, ob die Daten erhalten bleiben. Wenn das Projekt eine Datenbanksicherung vorsieht, verwenden Sie einen definierten isolierten Wiederherstellungsfall. Dokumentieren Sie den tatsächlich geprüften Code- und Datenstand und kennzeichnen Sie offen, was noch nicht ausgeführt wurde.

Ein Fehler bei der Suche nach Person B wird zunächst technisch diagnostiziert. Eine falsche Sortierung, eine nicht behandelte Kodierung oder eine fehlerhafte Abfrage kann die Ursache sein. I3 repariert sie innerhalb des bestehenden Charters und führt die relevanten Regressionen erneut aus.

Vor einer positiven Abnahme zeigt die Verwaltungskraft in einer echten Bedienprobe, ob sie die vorgesehenen Schritte ohne Spezialwissen versteht. Ist die Oberfläche verwirrend, kann die Anforderung fachlich unzureichend erfüllt sein, obwohl alle Tabellen korrekt sind.

10. Einen Befund an die richtige Rolle routen

Stellen Sie sich drei unterschiedliche Befunde vor. Erstens: „Das Speichern schlägt bei Umlauten fehl.“ Dies ist gewöhnlich ein technischer Defekt; I3 untersucht und behebt ihn. Zweitens: „Wir brauchen doch Selbstregistrierung über das Internet.“ Dies ist eine neue Produktabsicht mit Sicherheitsfolgen; der CLIENT bestätigt sie und der Planner bewertet den neuen Umfang. Drittens: „Wir wissen nicht, ob die Datenbank bereits eine Zahlung verbucht hat.“ Dies ist eine Currentness- und mögliche Idempotenzfrage; weitere schreibende Aktionen bleiben bis zur Rekonstruktion gesperrt.

Die richtige Route hängt also nicht allein davon ab, ob der Bericht alarmierend klingt. Sie hängt von Ursache, betroffenem Gate, bestehender Autorisierung und materiellen Folgen ab. Das System soll den CLIENT vor technischer Mikromanagementarbeit schützen, ohne ihn aus echten Produktentscheidungen auszuschließen.

Wenn ein technischer Defekt zum dritten Mal auftritt, wird die bisherige Symptombekämpfung hinterfragt. I3 prüft das ganze betroffene Subsystem und fordert eine unabhängige Überprüfung. Ein neuer Planner-Zyklus allein wegen wiederkehrender Implementierungsfehler würde die falsche Verantwortung belasten.

11. Zweiter Zyklus mit klaren Abhängigkeiten

Die Beitragsfunktion verwendet die zuvor angenommene Mitgliederidentität. Bevor I3 eine neue Zahlung schreibt, prüft er, dass diese Identität im tatsächlich aktuellen Kandidaten vorhanden und gültig ist. Falls die erste Fähigkeit seit ihrer Annahme verändert wurde, kann eine erneute Integrationsprüfung nötig sein.

Ein fiktiver Beitrag von 25 Einheiten wird für Person A erfasst. Anschließend liest die Anwendung den Datensatz wieder. Der Test wiederholt denselben fachlichen Speichervorgang und kontrolliert, dass keine doppelte Buchung entsteht. Für Person C wird die Deaktivierung geprüft: Historische Zahlungen bleiben auffindbar, während neue Vorgänge nach der bestätigten Regel eingeschränkt werden.

Der Controller legt einen konkreten Kandidaten und seine Nachweise vor. Ein Reviewer untersucht Grenzfälle, Datenintegrität und Fehlerbehandlung. Der CLIENT beurteilt die Verständlichkeit des Gesamtablaufs. Falls das Ergebnis die vereinbarten Bedingungen erfüllt, wird die Fähigkeit nachvollziehbar angenommen. Falls nicht, werden konkrete Fehler innerhalb des bestehenden Charters korrigiert.

Die nächste Fähigkeit könnte ein monatlicher Beitragsbericht sein. Sie wird erst sinnvoll geplant, wenn die tatsächliche Bedeutung von „bezahlt“, „offen“ und „überfällig“ bestätigt ist. Ein technischer Bericht ohne diese Fachregeln könnte mathematisch perfekt und für den Verein trotzdem nutzlos sein.

12. Übung: Fortsetzung in einem neuen Kontext

Für den Abschluss dieses Handbuchs legen Sie das ursprüngliche Gespräch beiseite. Bitten Sie eine neue Person oder Sitzung, ausschließlich anhand des aktuellen Anwendungsrepositorys folgende Fragen zu beantworten:

1. Wer benutzt die Anwendung, und welche Probleme soll sie zuerst lösen?
2. Welche Produktentscheidungen wurden ausdrücklich bestätigt?
3. Welcher reale Git- und Datenbankstand ist aktuell?
4. Welche Mitglieder- und Beitragsfunktionen sind angenommen?
5. Welche Tests, Reviews und echten Bedienproben belegen diese Annahme?
6. Welche Risiken, offenen Fragen und externen Freigaben fehlen?
7. Welche Kampagne ist noch aktiv, und wer darf sie fortsetzen?
8. Was darf die neue Sitzung als Nächstes tun, ohne nach der gesamten alten Chatgeschichte zu fragen?

Kann die Sitzung diese Punkte rekonstruieren, besitzt das Projekt einen belastbaren Übergang. Kann sie es nicht, ist die konkrete Lücke zu beheben: fehlende Anforderung, unklare Datenbankreferenz, nicht aktueller Review oder fehlende Ownership. Eine neue schöne Zusammenfassung ohne gültige Quellen würde das Grundproblem nicht lösen.

Der letzte praktische Test ist immer die reale Nutzbarkeit. Wenn die Verwaltung eine Person und ihre Beiträge zuverlässig verwalten kann, die Risiken kontrolliert und die Fortsetzung nachvollziehbar ist, hat das Framework seinen Zweck erfüllt. Es ist kein Selbstzweck und kein Ersatz für gute Softwareentwicklung.

Weiterführende Quellen

- Referenzprojekt und Transfer

- Öffentliche v1.4-Installation
- Bestehende Projekte prüfen
- Produktentdeckung
- Adoption
- Charter-Standard
- Capability-Gates und Evidenz
- Rekonstruktion aktueller Zustände

Das Vereinsbeispiel ist ein didaktisches Szenario. Seine vorgeschlagenen Anforderungen, Tests und Annahmen sind keine Aussage über den tatsächlichen Zustand eines existierenden Vereins.